



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA DE
SISTEMAS INFORMÁTICOS
GRADO EN INGENIERÍA DEL SOFTWARE

TRABAJO FIN DE GRADO

**Diseño e implementación de una arquitectura de Business Intelligence para
e-commerce basada en Data Warehouse y análisis avanzado de clientes**

AUTOR: Rodrigo García Arroyo
TUTOR: Carlos Camacho Gómez

Madrid, 2026

Resumen

El crecimiento del comercio electrónico ha generado grandes volúmenes de datos que permiten analizar en profundidad el comportamiento de los clientes y mejorar la toma de decisiones empresariales. En este contexto, las soluciones de Business Intelligence y Data Science resultan fundamentales para transformar datos en información útil. El objetivo de este Trabajo Fin de Grado es diseñar e implementar una arquitectura de Business Intelligence aplicada a un entorno de e-commerce, utilizando como base el dataset Brazilian E Commerce Public Dataset by Olist. Se busca construir una solución completa que permita estructurar, analizar y visualizar los datos, así como obtener información relevante sobre el funcionamiento del negocio. Para ello, en primer lugar se ha diseñado un modelo dimensional basado en la metodología de Kimball, definiendo un esquema en estrella que organiza la información en tablas de hechos y dimensiones. Este modelo permite analizar el negocio desde distintas perspectivas, como productos, clientes o tiempo. A partir de este diseño, se han desarrollado procesos de integración y transformación de datos mediante técnicas ETL, que permiten preparar y cargar la información en el Data Warehouse de forma adecuada. Sobre esta base, se han construido distintos cuadros de mando interactivos que permiten analizar aspectos claves del negocio, facilitando la interpretación de los datos y proporcionando una visión global del sistema. Adicionalmente, se han incorporado técnicas de análisis de datos con el objetivo de profundizar en el estudio de comportamiento del cliente y los patrones de compra. Los resultados obtenidos evidencian el valor de estructurar y analizar los datos de forma adecuada para comprender el funcionamiento del negocio. La solución desarrollada permite integrar información procedente de distintas fuentes y transformarla en conocimiento útil, facilitando el análisis de los principales indicadores y la identificación de oportunidades de mejora. En conjunto, el proyecto pone de manifiesto la importancia de las soluciones de Business Intelligence en entornos de comercio electrónico, proporcionando una base sólida para la toma de decisiones y para el desarrollo de futuros análisis más avanzados.

Abstract

The growth of electronic commerce has generated large volumes of data that make it possible to analyze customer behavior in depth and improve business decision-making. In this context, Business Intelligence and Data Science solutions are essential for transforming data into useful information. The objective of this Final Degree Project is to design and implement a Business Intelligence architecture applied to an e-commerce environment, using the Brazilian E-Commerce Public Dataset by Olist as a basis. The aim is to build a complete solution that enables the structuring, analysis, and visualization of data, as well as the extraction of relevant insights into business performance. To achieve this, a dimensional model based on the Kimball methodology has been designed, defining a star schema that organizes information into fact and dimension tables. This model allows the business to be analyzed from different perspectives, such as products, customers, or time. Based on this design, data integration and transformation processes have been developed using ETL techniques, enabling the proper preparation and loading of data into the Data Warehouse. On this basis, several interactive dashboards have been developed to analyze key aspects of the business, facilitating data interpretation and providing an overall view of the system. Additionally, data analysis techniques have been incorporated to further explore customer behavior and purchasing patterns. The results obtained demonstrate the value of properly structuring and analyzing data to understand how the business operates. The developed solution enables the integration of information from different sources and its transformation into useful knowledge, facilitating the analysis of key performance indicators and the identification of improvement opportunities. Overall, the project highlights the importance of Business Intelligence solutions in e-commerce environments, providing a solid foundation for decision-making and for the development of more advanced future analyses.

Agradecimientos

En primer lugar, me gustaría agradecer a mi tutor, D. Carlos Camacho Gómez, por su orientación y apoyo durante el desarrollo del Trabajo Fin de Grado. Su dedicación y disponibilidad han sido fundamentales para poder llevar a cabo este proyecto. Continúo dando las gracias a todo el personal docente del Grado en Ingeniería del Software por contribuir de forma positiva a mi formación a lo largo de estos años, así como a mis compañeros, que han hecho de esta etapa universitaria una experiencia para toda la vida tanto a nivel personal como académico. Por último, quiero agradecer especialmente a mi familia por su apoyo incondicional durante todo el proceso. Gracias a ellos he podido afrontar este reto con esfuerzo y constancia, valores que han sido clave para alcanzar este objetivo.

Índice

1. Introducción	12
1.1. Objetivos	12
2. Contextos y fundamentos	13
2.1. Business Intelligence en entornos de datos	13
2.2. Modelos de almacenamiento de datos	14
2.2.1. Data Warehouse	14
2.2.2. Modelo dimensional	16
2.3. Herramientas de análisis y visualización	18
2.3.1. Microsoft Power BI	18
2.3.2. Tableau	18
2.3.3. Qlik	19
2.3.4. Looker	19
2.4. Frameworks y lenguajes de programación	20
2.4.1. Python	20
2.4.2. R	21
2.4.3. D3.js	21
2.4.4. SQL	22
2.5. Introducción al análisis de datos	23
2.6. Justificación de las herramientas elegidas	24
2.6.1. Python	25
2.6.2. MySQL y MySQL Workbench	25
2.6.3. PowerBI Desktop	25
3. Descripción del problema y requisitos	27
3.1. Descripción del Dataset	27
3.1.1. olist_orders_dataset.csv	28
3.1.2. olist_order_items_dataset.csv	29
3.1.3. olist_customers_dataset.csv	29
3.1.4. olist_products_dataset.csv	30
3.1.5. olist_order_payments_dataset.csv	30
3.1.6. olist_order_reviews_dataset.csv	31
3.1.7. product_category_name_translation.csv	31
3.1.8. Archivos CSV descartados	31
4. Diseño de la solución	33
4.1. Diseño de la arquitectura del sistema	33
4.1.1. Data Sources	33
4.1.2. Data Area	34
4.1.3. Data Access Tools	34
4.1.4. Justificación de la ausencia de Área de Staging	35
4.2. Diseño del modelo dimensional	35
4.2.1. Metodología de diseño: Enfoque Kimball	35
4.2.2. Tipo de esquema: Galaxy Schema	36
4.2.3. Resumen de tablas del modelo	37
4.2.4. Decisiones generales de diseño	38
4.2.5. Uso de claves sustitutas (Surrogate Keys)	39
4.2.6. Diseño tablas dimensión	39

4.2.7. Diseño tablas hecho	41
4.3. Diseño de los dashboards	42
4.3.1. Boceto previo	42
4.3.2. Principios de diseño aplicados	43
4.3.3. Identidad visual	44
4.3.4. Selección de visualizaciones	44
4.3.5. Sistema de navegación	48
5. Implementación de la solución desarrollada	49
5.1. Integración y preparación de datos (ETL)	49
5.1.1. load_dim_customer.py	49
5.1.2. load_dim_product.py	51
5.1.3. load_dim_date.py	54
Referencias	57

Índice de figuras

1.	Evolución de la importancia del Business Intelligence a lo largo de los años.[1]	13
2.	Diagrama componentes Data Warehouse.[3]	15
3.	Ejemplo de esquema en estrella en un modelo dimensional. (Elaboración propia)	17
4.	Plataforma Power BI.[5]	18
5.	Plataforma Tableau.[6]	19
6.	Plataforma Qlik.[7]	19
7.	Plataforma Looker.[8]	20
8.	Plataforma Python.[9]	21
9.	Plataforma R.[10]	21
10.	Plataforma D3.[11]	22
11.	Plataforma SQL.[12]	23
12.	Relación entre los distintos ficheros del dataset Brazilian E-Commerce Public Dataset by Olist.[14]	28
13.	Arquitectura por capas del sistema desarrollado. (Elaboración propia)	33
14.	Modelo dimensional del sistema basado en un esquema en galaxia. (Elaboración propia)	37
15.	Boceto inicial de la estructura de los cuadros de mando. (Elaboración propia)	43
16.	Visualizaciones disponibles en Power BI para la construcción de cuadros de mando.[15]	48
17.	Configuración de conexión entre el proceso ETL y el Data Warehouse. (Elaboración propia)	49
18.	Fragmento de código correspondiente a la fase de extracción de <code>dim_customer</code> . (Elaboración propia)	50
19.	Fragmento de código correspondiente a la fase de transformación de <code>dim_customer</code> . (Elaboración propia)	50
20.	Fragmento de código correspondiente a la fase de carga de <code>dim_customer</code> . (Elaboración propia)	51
21.	Fragmento de código correspondiente a la fase de extracción de <code>dim_product</code> . (Elaboración propia)	52
22.	Fragmento de código correspondiente a la primera fase de transformación de <code>dim_product</code> . (Elaboración propia)	53
23.	Fragmento de código correspondiente a la segunda fase de transformación de <code>dim_product</code> . (Elaboración propia)	53
24.	Fragmento de código correspondiente a la fase de carga de <code>dim_product</code> . (Elaboración propia)	54
25.	Fragmento de código correspondiente a la fase de extracción de <code>dim_date</code> . (Elaboración propia)	55
26.	Fragmento de código correspondiente a la fase de transformación de <code>dim_date</code> . (Elaboración propia)	56
27.	Fragmento de código correspondiente a la fase de carga de <code>dim_date</code> . (Elaboración propia)	56

Índice de tablas

1.	Resumen de tablas del modelo dimensional	38
2.	Tabla <code>dim_customer</code>	39
3.	Tabla <code>dim_product</code>	40
4.	Tabla <code>dim_date</code>	40
5.	Tabla <code>fact_sales</code>	41
6.	Tabla <code>fact_payment</code>	42
7.	Tabla <code>fact_review</code>	42
8.	Justificación de las visualizaciones del dashboard de Visión General	45
9.	Justificación de las visualizaciones del dashboard de Productos y Categorías	45
10.	Justificación de las visualizaciones del dashboard de Clientes y Geografía	45
11.	Justificación de las visualizaciones del dashboard de Análisis de pagos	46
12.	Justificación de las visualizaciones del dashboard de Segmentación de clientes	46
13.	Justificación de las visualizaciones del dashboard de Predicción de satisfacción	47
14.	Resumen del script <code>load_dim_customer.py</code>	49
15.	Resumen del script <code>load_dim_product.py</code>	51
16.	Resumen del script <code>load_dim_date.py</code>	55

Acrónimos

BI Business Intelligence

DW Data Warehouse

ETL Extract, Transform and Load

KPI Key Performance Indicator

OLAP Online Analytical Processing

DS Data Science

DAX Data Analysis Expressions

LOD Level of Detail Expressions

CRM Customer Relationship Management

DOM Document Object Model

SVG Scalable Vector Graphics

HTML HyperText Markup Language

CSS Cascading Style Sheets

SQL Structured Query Language

SGBD Sistema de Gestión de Bases de Datos

API Application Programming Interface

1. Introducción

En la actualidad, el crecimiento del comercio electrónico ha generado grandes volúmenes de datos que, sin un tratamiento adecuado, carecen de valor. En este contexto, las herramientas de Business Intelligence permiten transformar estos datos en información útil mediante su análisis y visualización. Este Trabajo de Fin de Grado tiene como objetivo diseñar y desarrollar un cuadro de mando orientado al comercio electrónico, tomando como punto de partida el dataset Brazilian E-commerce Public Dataset by Olist. A partir de estos datos, se construye una solución que permite analizar aspectos clave del negocio como las ventas, los clientes o los métodos de pago. Para ello, primero se definió un modelo de datos y se trabajó en los procesos necesarios para limpiar y preparar la información antes de analizarla. A partir de ahí, se desarrollaron distintas visualizaciones interactivas que hacen que interpretar los datos sea bastante más sencillo e intuitivo. Además, el proyecto incorpora un componente de análisis de datos que permite profundizar en el comportamiento del cliente y del negocio, explorando patrones y relaciones dentro de los datos. En definitiva, este trabajo muestra cómo la combinación de técnicas de Business Intelligence y análisis de datos permite extraer valor de la información disponible y mejorar la comprensión del negocio.

1.1. Objetivos

En los últimos años, el crecimiento del comercio electrónico ha facilitado a las empresas recoger enormes cantidades de datos correspondientes a pedidos, clientes y productos entre otros muchos. A pesar de esto, todos estos datos no aportan ninguna funcionalidad si no se dispone de herramientas que permitan analizarlos y extraer información que permita entender el funcionamiento del negocio. En este ámbito, resulta indispensable incluir soluciones de Business Intelligence que favorezcan el análisis y la visualización de la información, especialmente utilizando cuadros de mando, que posibilitan representar de forma clara la información principal del sistema. Los principales objetivos de este trabajo fin de grado son:

- Diseñar un modelo dimensional basado en un esquema en galaxia (combinación de múltiples esquemas estrella) que permita estructurar la información para su análisis.
- Desarrollar procesos ETL que preparen la información para su explotación.
- Construir un Data Warehouse que sirva como base para el análisis.
- Crear dashboards interactivos en Power BI que faciliten la interpretación de datos.
- Analizar aspectos del negocio como los clientes, ventas o los métodos de pago, entre otros.
- Incorporar técnicas de Data Science que permitan un análisis más avanzado del negocio.

2. Contextos y fundamentos

En este punto se presentan los principales conceptos y fundamentos teóricos para entender las tecnologías utilizadas en este proyecto y su correspondiente funcionamiento.

2.1. Business Intelligence en entornos de datos

El Business Intelligence (BI) agrupa un conjunto de procesos, tecnologías y distintas metodologías enfocadas a recopilar, integrar y analizar datos dentro de una organización. A través del uso de diversas técnicas y herramientas, el Business Intelligence permite organizar los millones de datos que se generan al día, analizarlos desde distintas perspectivas y presentarlos de manera fácilmente entendible. Esto ayuda a los usuarios a extraer conclusiones relevantes y a respaldar la toma de decisiones, ya sea en el ámbito operativo o estratégico.

El Business Intelligence tiene sus orígenes en la mitad del siglo XX, cuando empezaron a desarrollarse los primeros sistemas enfocados al análisis de datos. El término fue introducido por Hans Peter Luhn en los años 50 y, a lo largo del tiempo, el concepto fue evolucionando gracias a los avances tecnológicos. Fue a partir de los años 90 cuando el BI comenzó a ganar importancia en el mundo empresarial, especialmente con la aparición de nuevas herramientas que facilitaron en gran escala el acceso y análisis de datos. Hoy en día las técnicas han avanzado aún más, integrando nuevas tecnologías que permiten trabajar con grandes volúmenes de datos y acceder a una visión más completa del negocio.

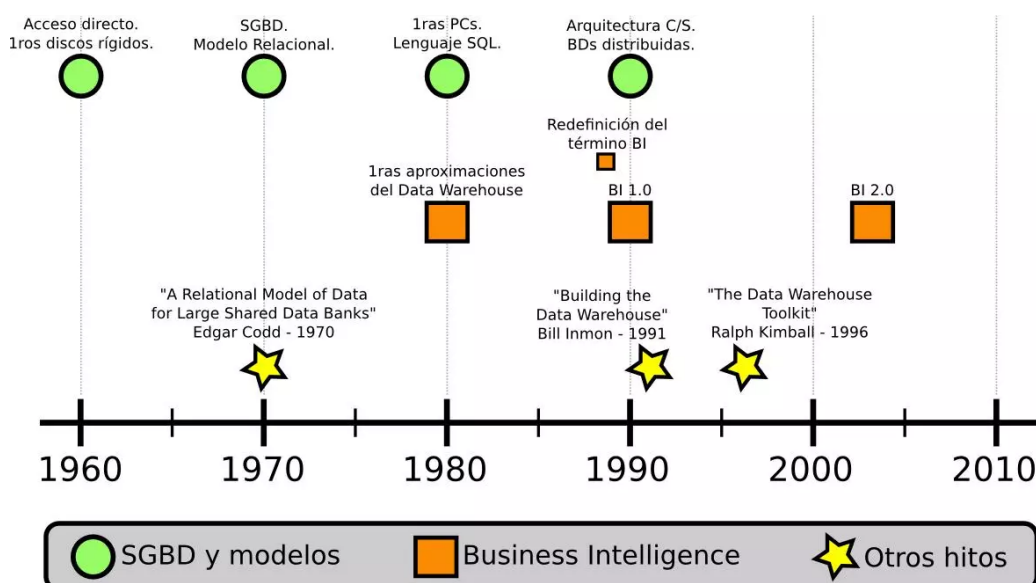


Figura 1: Evolución de la importancia del Business Intelligence a lo largo de los años.[1]

Por un lado, en el ámbito del análisis de datos, pueden distinguirse distintos enfoques en función del objetivo que se desea alcanzar:

- **Análisis descriptivo.** Se centra en examinar datos históricos con el objetivo de comprender qué ha sucedido, identificando tendencias y patrones.

- **Análisis diagnóstico.** Permite profundizar en los datos para entender por qué han ocurrido determinados resultados.
- **Análisis predictivo.** Emplea modelos estadísticos y técnicas analíticas para anticipar posibles comportamientos futuros.
- **Análisis prescriptivo.** Sugiere acciones o decisiones basadas en los resultados obtenidos en los análisis anteriores.

Por otro lado, el funcionamiento general del Business Intelligence se basa en la capacidad de analizar los datos y representarlos visualmente para facilitar su comprensión. Mediante el uso de gráficos, tablas y cuadros de mando, los usuarios pueden interpretar la información de forma más rápida y detectar patrones relevantes.

En este contexto, se pueden definir algunos conceptos clave:

- **Dashboard.** Herramienta de visualización que permite centralizar, organizar y mostrar los principales datos, indicadores y métricas del negocio en una única vista.
- **KPI (Key Performance Indicator).** Indicador o medida cuantificable que utiliza una empresa para evaluar el rendimiento, la eficiencia o el éxito de un proceso dentro del negocio, facilitando la toma de decisiones basada en datos objetivos.
- **Insight.** Hallazgo o conclusión relevante extraída a partir del análisis de datos que aporta valor al negocio.

2.2. Modelos de almacenamiento de datos

Dentro del ámbito del Business Intelligence, con el objetivo de analizar los datos de forma eficiente, es imprescindible disponer de modelos de almacenamiento para organizar los datos estructuradamente facilitando su análisis. En este apartado se definen las principales soluciones utilizadas en este proyecto.

2.2.1. Data Warehouse

Uno de los elementos fundamentales dentro de este ámbito es el Data Warehouse, cuyo objetivo principal es proporcionar un entorno adecuado para el almacenamiento y análisis de grandes volúmenes de datos. A diferencia de los sistemas operacionales, diseñados para gestionar transacciones diarias, un Data Warehouse está orientado específicamente al análisis y consulta de información.

En términos generales, puede entenderse como un repositorio centralizado que integra datos procedentes de distintas fuentes, organizándolos para facilitar su explotación. En esta línea, Golfarelli y Rizzi lo definen como “un repositorio de datos históricos, integrados y consistentes” [2].

Una de las características más destacadas de un Data Warehouse es su orientación al análisis. Esto significa que los datos que se almacenan no se están modificando permanentemente, sino que se mantienen constantes para permitir consultas eficientes. Además de esto, la información se encuentra integrada y organizada coherentemente, lo que facilita su uso por herramientas analíticas. A lo largo del tiempo se han propuesto y diseñado distintas opciones para el diseño de un Data Warehouse. Por

un lado, algunos proponen integrar todos los datos desde un principio, mientras que otros proponen un desarrollo incremental por áreas. Estos distintos enfoques permiten que cada organización adapte el desarrollo a sus intereses y necesidades. En cuanto al punto de vista estructural, se compone de distintos elementos que permiten gestionar el flujo de datos desde que se originan hasta su explotación. Entre los más destacados se encuentran:

- **ODS (Operational Data Store).** Es un sistema diseñado para integrar información de distintas fuentes en un entorno unificado. Su función principal es actuar de intermediario entre los sistemas operacionales y el Data Warehouse, facilitando el acceso a una versión integrada previa al almacenamiento definitivo.
- **Staging Area.** Es otro de los componentes intermediarios dentro de la arquitectura de un Data Warehouse. Su función principal es actuar como espacio temporal donde tienen lugar las tareas de extracción y transformación de los datos antes de su carga. Este sistema se diferencia del resto en que los datos almacenados no son persistentes, sino que suelen eliminarse una vez terminada la tarea asociada.
- **Datamarts.** Son subconjuntos de datos pertenecientes a un Data Warehouse orientados a un ámbito específico. En este caso en lugar de almacenar toda la información disponible, se enfoque solo en un bloque, por ejemplo, un departamento de ventas. Esto permite simplificar el acceso a la información y adaptarse únicamente a las necesidades específicas de esa área.
- **Metadatos.** Los metadatos son otros de los elementos fundamentales en un Data Warehouse. Básicamente proporcionan información sobre los datos almacenados con el objetivo de facilitar su comprensión. Se pueden entender como un mapa para gestionar el flujo de datos desde su fase inicial hasta los informes finales.

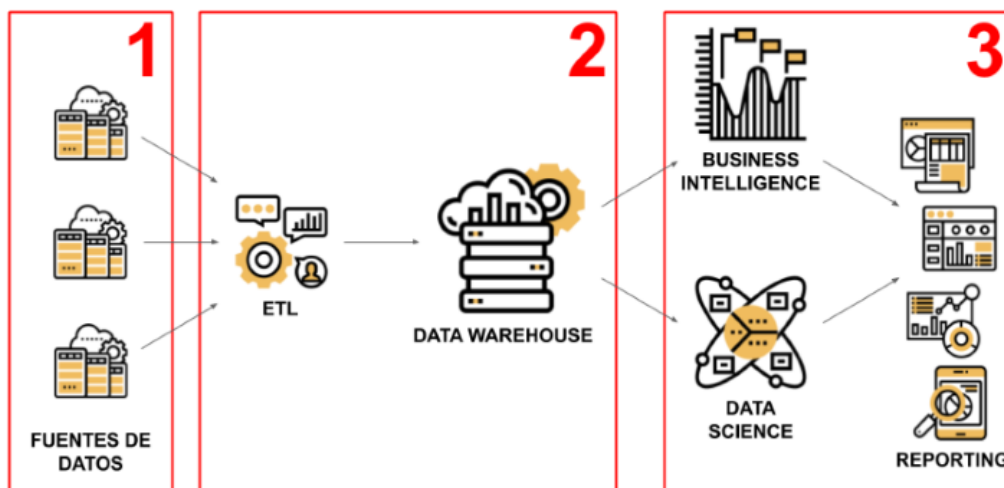


Figura 2: Diagrama componentes Data Warehouse.[3]

2.2.2. Modelo dimensional

El modelo dimensional es una técnica de diseños de bases de datos enfocada al análisis y consulta de grandes volúmenes de datos. Este esquema se diferencia del modelo tradicional en que no se centra en principalmente en eliminar la redundancia y garantizar la integridad de los datos, sino que apuesta por facilitar la comprensión por parte del usuario final y el rendimiento a la hora de ejecutar consultas complejas. Esta característica lo convierte en el estándar principal en el diseño de Data Warehouse y sistemas de Business Intelligence.

Este concepto fue normalizado por Ralph Kimball, cuya metodología conocida como “el enfoque bottom up” tiene como objetivo diseñar el almacén de datos a partir de subconjuntos de datos independientes orientados a un ámbito, es decir, los denominados Datamarts[4]. Kimball fija que todo modelo dimensional se diseña en torno a dos tipos de tablas: las tablas de hechos o fact tables y las tablas de dimensiones o dimensión tables.

Las tablas de hechos constituyen el núcleo del modelo dimensional. Almacenan los eventos del negocio que se desean analizar: ventas, pedidos. . . y están formadas principalmente por medidas cuantificables, denominadas métricas, que permiten facilitar el análisis. Cada registro en la tabla de hechos representa una ocurrencia de negocio específica en un nivel determinado de granularidad. La granularidad, por su parte, es uno de los conceptos más críticos y se encarga de determinar el grado de detalle con el que se registra cada hecho y condiciona directamente la flexibilidad del modelo. Por otro lado, otro concepto importante que contienen las tablas de hechos son las claves foráneas, encargadas de referenciar a las tablas de dimensiones asociadas con el objetivo de contextualizar el análisis.

Las tablas de dimensiones, por su parte, aportan el contexto descriptivo en torno a cada hecho. Presentan atributos que permiten clasificar y dividir los datos, como por ejemplo el nombre de un producto o su región geográfica. En comparación con las tablas de hechos, estas suelen tener un menor tamaño, pero con una gran riqueza descriptiva en sus columnas. Un aspecto de gran importancia en el diseño es el tratamiento de los cambios en las dimensiones a lo largo del tiempo, conocidos como Slowly Changing Dimensions (SCD). Kimball clasifica estos cambios en tres grandes tipos:

- **Tipo 1 (Sobreescritura).** Es la estrategia más sencilla. Se basa en que cuando un atributo de una dimensión cambia, el valor se sustituye automáticamente, es decir, siempre se mantienen los datos actualizados y no se guarda ningún dato histórico. De aquí sale el principal inconveniente de este tipo, se pierde toda la trazabilidad histórica, pudiendo distorsionar el análisis retrospectivo.
- **Tipo 2 (Historial mediante nuevas filas).** Es el más utilizado en entornos de BI ya que ofrece una mayor riqueza analítica. Cuando se produce un cambio, no se modifica el registro, sino que se añade una nueva fila en la dimensión con el nuevo valor del atributo, dejando intacto el registro actual. Para la gestión de cada versión habitualmente se incluyen tres columnas adicionales: una fecha de inicio de validez, una fecha de fin de validez y un indicador booleano que indica si el valor corresponde al más actual. Debido a que cada versión genera un nuevo registro con clave primaria diferente, los hechos históricos permanecen vinculados a la versión vigente en el momento que ocurrieron, lo que permite por ejemplo calcular las ventas de un cliente cuando pertenecía a un determinado departamento y compararlas con las generadas en la actualidad. La principal desventaja de esta estrategia es el gran aumento de volumen que supone a lo largo del

tiempo, así como un aumento en la complejidad de las consultas.

- **Tipo 3 (Atributo adicional para el valor anterior).** Esta estrategia mantiene un enfoque intermedio. En lugar de añadir filas, en una misma fila añade columnas adicionales para mantener el registro tanto del valor actual como del anterior. Esta estrategia puede resultar útil cuando la empresa necesita comparar el estado actual y anterior de forma habitual y directa. La limitación clara de este sistema es que solo permite guardar un número fijo de versiones anteriores, por lo que será insuficiente en un sistema donde los atributos varíen con una alta frecuencia.

La disposición física de estas tablas da lugar al esquema en estrella, que recibe nombre debido a que, a la hora de representarlo, las tablas de hechos ocupan el centro y las dimensiones se dirigen hacia afuera dando la sensación visual de una estrella. Esta estructura permite responder a preguntas analíticas de manera eficiente, por ejemplo, si tenemos una tabla de hechos FactVentas y una tabla de dimensión DimTiempo, podremos responder a preguntas como ¿Cuánto se vendió en un periodo de tiempo concreto? Es la técnica más típica y recomendada para implementar un modelo dimensional, ya que aporta un equilibrio entre simplicidad y eficiencia. Una variante del esquema en estrella es el esquema en copo de nieve, que consiste en descomponer los atributos de las dimensiones en otras tablas para reducir la redundancia de datos. Sin embargo, esto introduce una mayor complejidad en las consultas además de afectar al rendimiento, por lo que se suele apostar más por el diagrama en estrella normalizado. Por otro lado encontramos el modelado en galaxia, el cual, a diferencia del modelado en estrella, incluye múltiples tablas de hechos que comparten dimensiones comunes.

En definitiva, el modelo dimensional y el esquema en estrella constituyen la base sobre la que se sustenta el Data Warehouse, facilitando todo el proceso desde el punto de vista de la comprensión como de la integración de futuras herramientas.

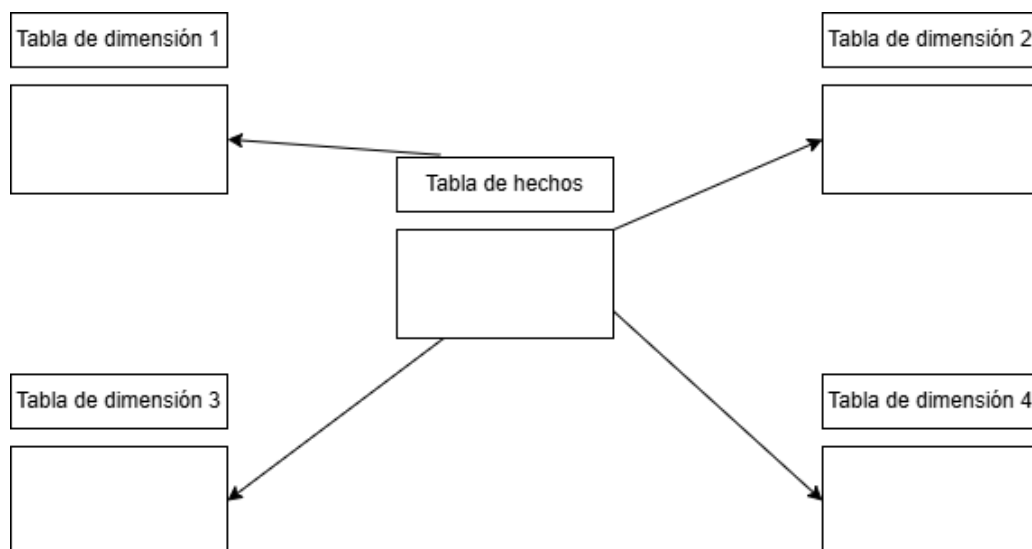


Figura 3: Ejemplo de esquema en estrella en un modelo dimensional. (Elaboración propia)

2.3. Herramientas de análisis y visualización

En el mundo del desarrollo de soluciones Business Intelligence encontramos un amplio rango de herramientas disponibles. Las herramientas de visualización constituyen la capa mas visible de las soluciones BI. Son las encargadas de transformar los datos procesados en representaciones gráficas que permiten al usuario final tratar con los datos.

2.3.1. Microsoft Power BI

Power BI es la plataforma de Business Intelligence desarrollada por Microsoft y una de las más destacadas a nivel mundial. Está compuesta por varios productos complementarios:

- **Power BI Desktop.** La aplicación de escritorio donde se modelan los datos y se construyen los informes.
- **Power BI Service.** El servicio en la nube que permite publicar, compartir y colaborar sobre los informes.
- **Power BI Mobile.** La aplicación para el uso en dispositivos móviles.

Su integración con el ecosistema Microsoft, incluyendo servicios como Azure, Excel o SQL Server, la convierte en una opción muy atractiva para las empresas que ya utilizan estos servicios. Destaca por su potente motor de modelado basado en DAX, su conectividad con un amplio catálogo de datos y su gran capacidad para actualizar informes tanto en tiempo real como de forma programada.



Figura 4: Plataforma Power BI.[5]

2.3.2. Tableau

Tableau es una de las herramientas de visualización de datos mas reconocidas en el ámbito del BI, especialmente por la calidad y flexibilidad de sus representaciones gráficas. Permite construir visualizaciones de alta complejidad sin necesidad de programación, aunque también ofrece un lenguaje propio llamado LOD para análisis más avanzados. Esta herramienta fue adquirida por la empresa Salesforce en 2019 y, desde ese momento, ha reforzado su integración con soluciones CRM y plataformas cloud. La principal desventaja de esta herramienta frente a otras más destacadas como Power BI es su elevado coste de licenciamiento, siendo un gran inconveniente para proyectos de menor escala.



Figura 5: Plataforma Tableau.[6]

2.3.3. Qlik

Qlik es una empresa de software líder en el sector de BI que ofrece dos productos diferenciados:

- **QlikView.** Orientado al desarrollo de aplicaciones analíticas guiadas.
- **QlikSense.** Herramienta más moderna y enfocada al autoservicio analítico.

Ambos se basan en un motor asociativo propio que permite explorar relaciones de datos en cualquier dirección sin definir previamente jerarquías, a diferencia de los sistemas tradicionales. Esta capacidad es una de sus principales características que lo diferencian a nivel competitivo.



Figura 6: Plataforma Qlik.[7]

2.3.4. Looker

Looker es una plataforma de BI basada en web que fue adquirida por Google en 2019 y actualmente se encuentra integrada en Google Cloud. Su principal distintivo es el uso de LookML, un lenguaje de modelo que centraliza la definición de métricas y dimensiones en una capa reutilizable. Gracias a esto se facilita la consistencia de los datos en la organización y se reduce la dispersión de definiciones en distintos equipos. Se enfoca en entornos cloud-native y empresas que contienen equipos de datos maduros.



Figura 7: Plataforma Looker.[8]

2.4. Frameworks y lenguajes de programación

Además de las herramientas y representaciones visuales, el desarrollo de las soluciones BI requiere el uso de lenguajes de programación y frameworks para automatizar los procesos de extracción, transformación y carga de datos (ETL), realizar análisis avanzados y gestionar la infraestructura de datos subyacente.

2.4.1. Python

Python es el lenguaje de referencia en el ámbito del análisis e ingeniería de datos. Su sintaxis clara y extenso contenido de bibliotecas lo hacen la mejor alternativa para realizar tareas de procesamiento y transformación de datos. Entre las bibliotecas mas importantes en contextos de BI destacan:

- **Pandas.** Manipulación y análisis de datos tabulares.
- **NumPy.** Operaciones numéricas y matriciales.
- **SQLAlchemy.** Interacción con bases de datos relacionales mediante una capa de abstracción orientada a objetos.
- **Matplotlib/Seaborn.** Generación de visualizaciones estáticas.

Adicionalmente encontramos otras bibliotecas como Airflow que permiten orquestar flujos de trabajo de datos complejos, mientras que otras como data build tool facilita la transformación de datos directamente en el Data Warehouse.



Figura 8: Plataforma Python.[9]

2.4.2. R

R es un lenguaje estadístico de código abierto utilizado en entornos académicos y de investigación. Ofrece capacidades de análisis estadístico y diferentes paquetes para la visualización de datos, siendo ggplot2 uno de los más utilizados. Pese a no tener el mismo reconocimiento en el ámbito empresarial que otros lenguajes como Python, es una opción válida y con gran potencia para proyectos que contienen componentes estadísticos o predictivos.



Figura 9: Plataforma R.[10]

2.4.3. D3.js

D3.js (Data-Driven Documents), desarrollado por Mike Bostock, es la biblioteca de referencia para la visualización de datos en entornos web. Permite vincular datos a elementos del DOM y aplicarles transformaciones visuales mediante SVG, HTML Y CSS, ofreciendo así un control total sobre el gráfico

resultante. Presenta una flexibilidad prácticamente ilimitada, lo que también implica la necesidad de una curva de aprendizaje pronunciada, ya que requiere una gran cantidad de conocimientos.



Figura 10: Plataforma D3.[11]

2.4.4. SQL

SQL es el lenguaje estándar para la definición, manipulación y consultas de datos en sistemas de bases de datos relacionales. Resulta fundamental en los proyectos de Data Warehouse, tanto para la creación del esquema de tablas como para la construcción de las transformaciones ETL y las consultas sobre el modelo dimensional.

Dentro de los distintos gestores que implementan este lenguaje, podemos destacar MySQL, un SGBD de código abierto que destaca por su estabilidad, rendimiento y su amplia comunidad de soporte. Además, dispone de MySQL Workbench como entorno gráfico oficial, que permite la creación y diseño de esquemas, la ejecución de consultas y la administración del servidor, lo que simplifica en gran medida las tareas de modelado y mantenimientos de la base de datos.



Figura 11: Plataforma SQL.[12]

2.5. Introducción al análisis de datos

El crecimiento exponencial en el volumen de datos generados por las empresas ha impulsado el desarrollo de una serie de técnicas orientadas a extraer valor de dichos datos: el Data Science o ciencia de datos. A pesar de que el análisis de datos como concepto lleva existiendo mucho tiempo atrás, fue a finales del siglo XXI cuando el término Data Science comenzó a ganar popularidad, siendo conocido como una combinación de técnicas que combinan estadística, matemáticas, programación y conocimiento del negocio con el objetivo de apoyar la toma de decisiones basadas en la evidencia.

En cuanto a la relación de Data Science y Business Intelligence, a pesar de compartir su intención de convertir datos en información útil, se diferencian en su enfoque y el tipo de preguntas que afrontan. Por un lado, el BI tradicional se orienta hacia el análisis descriptivo, haciendo uso de herramientas que monitorizan el rendimiento del negocio mediante indicadores y cuadros de mando. Por otro lado, el Data Science amplía este alcance incorporando técnicas de mayor complejidad. Dentro de este ámbito, podemos distinguir varios tipos de análisis:

- **Análisis diagnóstico.** Trata de explicar por qué ha ocurrido algo.
- **Análisis predictivo.** Estima que es probable que ocurra en el futuro aplicando modelos estadísticos y de aprendizaje automático.
- **Análisis prescriptivo.** Va un paso más allá que el resto de los análisis, sugiriendo qué acción debería tomarse para optimizar un resultado.

En la práctica, ambas disciplinas son complementarias y suelen utilizarse conjuntamente, siendo común que los equipos de Data Science integren sus modelos como una capa más sobre la infraestructura BI existente.

A partir de lo mencionado anteriormente, resulta fundamental comprender el ciclo de vida que siguen los datos en este tipo de proyectos.

En primer lugar, todo comienza con la recopilación de los datos, que se basa en identificar toda la información accediendo a las distintas fuentes: bases de datos, APIs, ficheros u otras fuentes. Tras tener toda la información recogida, debemos proceder a la limpieza de los datos, fase fundamental y en la que probablemente emprendamos más tiempo. Durante esta fase, se detectan posibles inconsistencias en las fuentes de datos, por ejemplo, valores nulos, errores en la sintaxis que pueden provocar fallos en un futuro, valores duplicados o simplemente información que no consideremos de importancia para nuestros futuros análisis, por lo que trataremos los datos hasta disponer de la información necesaria para nuestro proyecto.

Una vez tengamos los datos preparados, debemos pasar a la fase de análisis, etapa muy importante en los proyectos de Data Science, en la que a través de la observación de los datos decidiremos cuáles son las técnicas más efectivas y adecuadas para nuestro proyecto. Posteriormente pasaremos a la fase de modelado, en la que aplicaremos los algoritmos correspondientes: regresión, clasificación, clustering, entre otros, para extraer patrones o construir modelos predictivos. Por último, finalizaremos el proceso con la fase de despliegue en la que transformaremos los resultados en representaciones visuales como informes o gráficas con el objetivo de facilitar el entendimiento por parte del usuario final.

En paralelo a esta evolución en el análisis de datos, se ha producido una aparición y especialización de distintos roles vinculados a este ámbito, entre los que destacan:

- **Data Engineer.** Es el responsable de diseñar y mantener la infraestructura de datos, por lo que su trabajo está vinculado a los pipelines ETL, almacenes de datos y sistemas de integración.
- **Data Analyst.** Se centra en la exploración y visualización de datos históricos para apoyar a la toma de decisiones, por lo que es el rol más cercano y vinculado al BI tradicional.
- **Data Scientist.** Combina conocimientos estadísticos y de programación para desarrollar modelos predictivos y técnicas de aprendizaje automático.
- **Machine Learning Engineer.** Se especializa en llevar los modelos desarrollados por el Data Scientist a entornos de producción, garantizando su escalabilidad y mantenimiento a lo largo del tiempo. Además de esto, también desarrolla sus propios modelos basados en datos.

En la realidad, estos roles no tienen una clara diferenciación, existiendo con frecuencia personas que asumen distintas responsabilidades de varios de ellos, especialmente en organizaciones que presentan un tamaño medio.

2.6. Justificación de las herramientas elegidas

El criterio de elección de herramientas para realizar este proyecto se ha basado en la experiencia previa utilizando ciertas tecnologías, así como en un análisis de las herramientas más adecuadas en los diferentes ámbitos tratados. A continuación se justifica la elección de cada herramienta en relación con su función dentro de la arquitectura de la solución.

2.6.1. Python

Python ha sido la herramienta central de este proyecto desde el punto de vista del desarrollo, teniendo una función esencial en dos aspectos clave. Por un lado, es el motor de los procesos ETL, encargado de extraer los datos, transformarlos según las necesidades del proyecto y posteriormente cargarlos para crear el Data Warehouse. Por otro lado, ha sido el entorno de implementación de los métodos de Data Science aplicados para complementar el análisis de los datos.

La elección de esta herramienta se justifica por varios motivos. En primer lugar, es una tecnología que he usado a lo largo de la carrera, teniendo una experiencia previa a la que, sumado el trabajo de investigación, me ha permitido adaptarme mejor al desarrollo. En segundo lugar, hay que destacar que Python presenta una enorme cantidad de bibliotecas, especialmente orientadas al tratamiento de datos, destacando Pandas para la manipulación de datos y mysql-connector-python para la integración con MySQL. En segundo lugar, valorar la versatilidad de la herramienta, la cual ha permitido efectuar en un mismo lenguaje tanto la ingeniería de datos como el análisis avanzado, evitando así la necesidad de incluir nuevas tecnologías en cada fase. Finalmente, Python es el lenguaje principal en el ámbito del aprendizaje automático y la ciencia de datos, por lo que su uso ha sido esencial para la implementación de algoritmos de clustering y predicción, que son parte del alcance de este trabajo.

2.6.2. MySQL y MySQL Workbench

MySQL ha sido el sistema gestor de bases de datos elegido para alojar el Data Warehouse del proyecto. La alta experiencia utilizando este sistema en distintas asignaturas como base de datos o base de datos avanzadas sumado a su naturaleza open source y su amplia contabilidad con Python lo convierte en una opción sólida para el almacenamiento y gestión de los datos transformados. Sobre MySQL se ha implementado el modelo dimensional en esquema galaxia, diseñado mediante MySQL Workbench, que integra las distintas tablas de hechos y dimensiones que conforman la base del proyecto.

La conexión entre Python y MySQL se ha realizado mediante la biblioteca mysql-connector-python, el conector oficial de MySQL para Python, que permite ejecutar desde los scripts ETL todas las operaciones necesarias sobre la base de datos: vaciado de tablas, inserciones masivas mediante `executemany` y consultas de dimensiones para la resolución de claves sustitutas.

Por otro lado, MySQL Workbench ha sido utilizado, además de para diseñar el modelo dimensional, para la administración de la base de datos resultante, permitiendo verificar y supervisar la estructura de las tablas. Por último, ha actuado como punto de conexión entre la capa de almacenamiento y la capa de visualización, facilitando la importación de los datos directamente en Power BI Desktop a través de la conexión con el servidor MySQL.

2.6.3. PowerBI Desktop

Power BI Desktop ha sido la herramienta elegida para la construcción de los cuadros de mando que conforman la capa de visualización de la solución. La elección de esta herramienta, en este caso, no se basa en la experiencia previa, lo que ha implicado la visualización de un curso para comprender las bases de la herramienta [13]. Su elección por tanto se ha basado en su capacidad para conectarse directamente a la base de datos de MySQL, importar el modelo dimensional definido y construir sobre

este un modelo semántico que permita a los usuarios finales explorar los datos de forma intuitiva. A este hay que añadir la recomendación previa de compañeros para utilizar esta herramienta debido a considerarla intuitiva y fácil de manejar.

Entre sus características más relevantes para este proyecto destaca su motor de cálculo basado en DAX, el cual permite definir métricas sobre el modelo dimensional. Por otro lado, también cabe destacar su amplio catálogo de visualizaciones interactivas y su capacidad para integrar en un mismo informe datos provenientes de distintas tablas del modelo, respetando las relaciones previamente definidas entre hechos y dimensiones. Además, hay que destacar también que la herramienta es de uso gratuito en su versión de escritorio, lo que la hace accesible sin coste adicional para el desarrollo del proyecto.

3. Descripción del problema y requisitos

En este apartado se presenta el contexto del problema abordado en el proyecto, los archivos en formato CSV con los datos empleados y los requisitos que debe satisfacer el sistema desarrollado.

En este contexto, el principal problema tratado radica en la ausencia de una estructura unificada que permita transformar los datos disponibles en información útil para la toma de decisiones por parte de los distintos actores del negocio, como gestores o vendedores. A pesar de que se dispone de gran cantidad de información relativa a pedidos, clientes, pagos y valoraciones, esta se dispone muy dispersa en tablas independientes y altamente normalizadas, lo que complica obtener una visión global de negocio.

Para dar respuesta a esta problemática, se plantea el diseño e implementación de una arquitectura de BI basada en un Data Warehouse, cuya implementación se desarrollará en apartados posteriores.

3.1. Descripción del Dataset

El sistema desarrollado en este trabajo utiliza el Brazilian E-Commerce Public Dataset by Olist, un conjunto de datos público disponible en la plataforma Kaggle [14]. Este dataset fue publicado por Olist, la mayor empresa de venta online en marketplaces brasileños, y contiene información real y anonimizada de aproximadamente 100.000 pedidos realizados entre los años 2016 y 2018 en múltiples plataformas de comercio electrónico de Brasil.

El dataset incluye nueve archivos CSV interrelacionados mediante identificadores comunes, entre los que destacan `order_id`, `customer_id` y `product_id`, y cubre todas las fases del ciclo de vida de un pedido de e-commerce : desde el momento de la compra hasta que es valorado por el cliente.

A continuación se describe cada uno de los archivos del dataset original, indicando para cada uno de ellos su contenido. Posteriormente se señala también los archivos que han sido descartados para el desarrollo del proyecto y la razón de esta decisión.

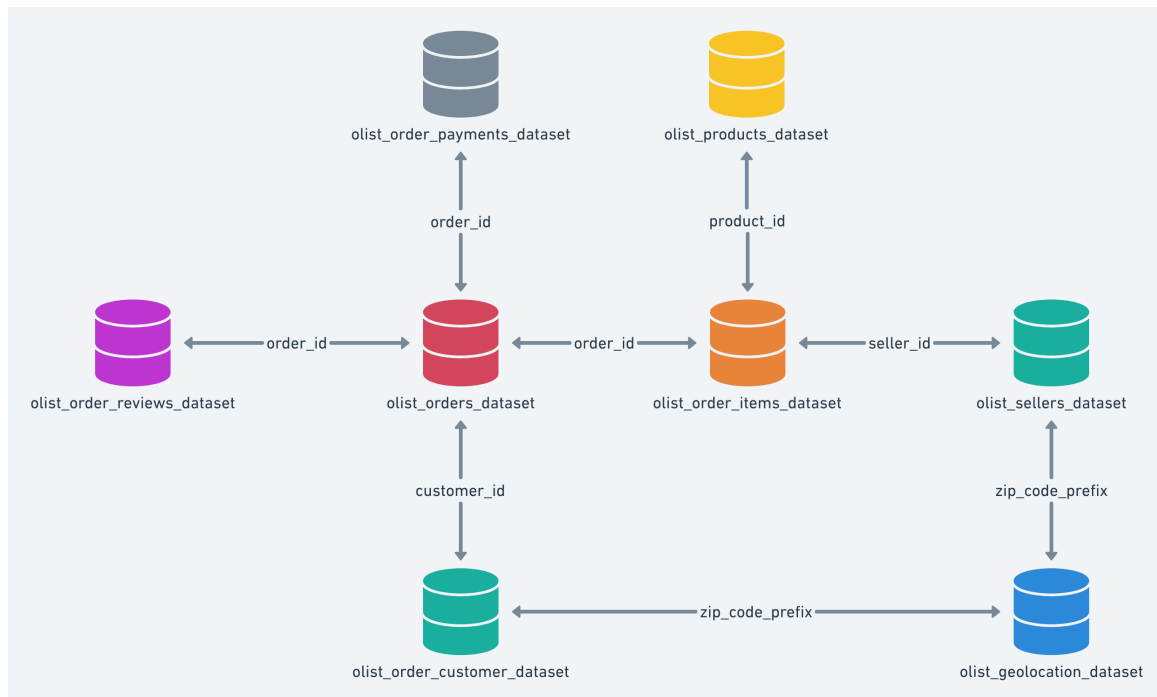


Figura 12: Relación entre los distintos ficheros del dataset Brazilian E-Commerce Public Dataset by Olist.[14]

3.1.1. olist_orders_dataset.csv

Este fichero contiene la información general de los pedidos realizados en la plataforma de comercio electrónico, así como las fechas asociadas a su ciclo de vida.

Su contenido viene dividido en las siguientes columnas:

- **order_id.** Identificador único del pedido.
- **customer_id.** Identificador del cliente asociado al pedido.
- **order_status.** Estado actual del pedido (delivered, shipped, canceled, invoiced, processing, unavailable y approved).
- **order_purchase_timestamp.** Fecha y hora en la que el pedido fue realizado.
- **order_approved_at.** Fecha y hora en la que el pedido fue aprobado por el sistema.
- **order_delivered_carrier_date.** Fecha y hora en la que el pedido fue entregado al personal de transporte.
- **order_delivered_customer_date.** Fecha y hora en la que el pedido fue entregado al cliente.
- **order_estimated_delivery_date.** Fecha estimada de entrega mostrada al cliente a la hora de la compra.

Este fichero, por tanto, es la base temporal y operativa para el análisis de ventas.

3.1.2. olist_order_items_dataset.csv

Este fichero describe los productos incluidos en cada pedido. Define la granularidad del hecho principal de ventas.

Su contenido viene dividido en las siguientes columnas:

- **order_id**. Identificador único del pedido al que pertenece el item.
- **order_item_id**. Identificador que permite distinguir múltiples productos en un mismo pedido.
- **product_id**. Identificador único del producto vendido.
- **seller_id**. Identificador único del vendedor que suministra el producto.
- **shipping_limit_date**. Fecha y hora límite para el envío del pedido al socio logístico por parte del vendedor.
- **price**. Precio del producto.
- **freight_value**. Coste de envío asociado al item.

Para calcular el precio final de un pedido puede haber confusiones, por lo que se debe tener en cuenta lo siguiente: Si tenemos un pedido con id 1, el cuál tiene asociado tres productos iguales con un precio de 20\$ y un coste de envío de 10\$, el valor total del pedido sería $20 \cdot 3 + 10 \cdot 3 = 90\$$.

3.1.3. olist_customers_dataset.csv

Este fichero contiene la información general de los clientes registrados en la plataforma de comercio electrónico.

Su contenido viene dividido en las siguientes columnas:

- **customer_id**. Identificador único del cliente asociado a un pedido. En este caso no es único, sino que un mismo cliente tendrá varios **customer_id** si realiza múltiples compras.
- **customer_unique_id**. Identificador único del cliente dentro de la plataforma.
- **customer_zip_code_prefix**. Prefijo del código postal del cliente.
- **customer_city**. Ciudad de residencia del cliente.
- **customer_state**. Región de residencia del cliente.

Este fichero es fundamental para identificar a cada usuario y asociarlo con su localización, facilitando los análisis y segmentaciones en función de las regiones o ciudades.

3.1.4. olist_products_dataset.csv

Este fichero contiene la información descriptiva de los productos registrados.

Su contenido viene dividido en las siguientes columnas:

- `product_id`. Identificador único del pedido.
- `product_category_name`. Nombre de la categoría del producto.
- `product_name_length`. Longitud del nombre del producto.
- `product_description_length`. Longitud de la descripción del producto.
- `product_photos_qty`. Número de fotografías asociadas al producto.
- `product_weight_g`. Peso del producto en gramo.
- `product_length_cm`. Longitud del producto en centímetros.
- `product_height_cm`. Altura del producto en centímetros.
- `product_width_cm`. Anchura del product en centímetros.

La principal característica de este archivo es que nos aporta la categoría del producto, lo que permite un futuro análisis en función de este dato.

3.1.5. olist_order_payments_dataset.csv

Este fichero contiene la información relativa a los pagos asociados a los pedidos realizados.

Su contenido viene dividido en las siguientes columnas:

- `order_id`. Identificador único del pedido al que corresponde el pago.
- `payment_sequential`. Número que identifica cada pago dentro de un mismo pedido. Es decir, un cliente puede pagar un pedido con más de un método de pago y, si lo hace, se incrementa el número.
- `payment_type`. Tipo de método de pago utilizado (credit card, boleto, voucher, entre otros).
- `payment_installments`. Número de cuotas en las que se ha dividido el pago.
- `payment_value`. Importe total del pago realizado.

Esta información es clave para el análisis financiero del negocio, permitiendo realizar distintos análisis como el comportamiento de pago de los clientes y los hábitos mas frecuentes.

3.1.6. `olist_order_reviews_dataset.csv`

Este fichero contiene la información sobre las valoraciones realizadas por los clientes en las encuestas de satisfacción. Cuando el cliente recibe el pedido o llega el día de la fecha estimada de recibirlo, el cliente recibe una encuesta por email donde pone su valoración.

Su contenido viene dividido en las siguientes columnas:

- `review_id`. Identificador único de la valoración.
- `order_id`. Identificador único del pedido al que está asociada la valoración.
- `review_score`. Puntuación otorgada por el cliente en una escala del 1 al 5.
- `review_comment_title`. Título del comentario de la valoración.
- `review_comment_message`. Contenido del comentario realizado por el cliente.
- `review_creation_date`. Fecha en la que se creó la valoración.
- `review_answer_timestamp`. Fecha y hora en la que se respondió la valoración.

Esta información es esencial para identificar patrones que influyen en la satisfacción del cliente y apoyar a la toma de decisiones para mejorar su experiencia.

3.1.7. `product_category_name_translation.csv`

Este fichero proporciona la traducción al inglés de la categoría del producto.

Su contenido viene dividido en las siguientes columnas:

- `product_category_name`. Nombre de la categoría en portugués.
- `product_category_name_english`. Traducción al inglés de la categoría.

Este fichero se utiliza para facilitar los posteriores análisis disponiendo de la información en inglés en lugar de en portugués, como viene dado por defecto.

3.1.8. Archivos CSV descartados

Durante la planificación del proyecto se ha decidido descartar dos archivos CSV de los 9 existentes en el dataset por distintas razones.

En primer lugar, `olist_sellers_dataset.csv` ha sido descartado debido a que la información relativa a la localización de los vendedores no ha sido considerada de interés en los análisis del proyecto y, además, la información del vendedor ya queda parcialmente representada a través del campo `seller_id` incluido en el archivo `olist_order_items_dataset.csv`, por lo que crear una dimensión adicional incrementaría la complejidad del modelo sin aportar un valor significativo.

Por otro lado, el otro archivo descartado ha sido `olist_geolocation_dataset.csv`, debido especialmente a su alta granularidad, ya que ofrece coordenadas geográficas a nivel de código postal. Además,

el archivo contiene más de un millón de registros, convirtiéndolo en un dato de escasa utilidad para los objetivos de este proyecto.

Estas decisiones permiten simplificar el modelo dimensional, adaptando el modelo a los principales objetivos de análisis del proyecto.

4. Diseño de la solución

4.1. Diseño de la arquitectura del sistema

El sistema de Business Intelligence desarrollado ha decidido organizarse en tres capas funcionales, las cuales pueden ser observada en la imagen. Estas capas son, en primer lugar, la capa que contiene los archivos csv del sistema origen, denominada Data Source. En segundo lugar, se encuentra el área de datos, denominada Data Area, que contiene la base de datos almacenada en torno al modelo dimensional. Por último, tenemos la capa de acceso y visualización, la cual contiene las representaciones visuales del análisis realizado de los datos. Cabe destacar también que, a diferencia de como suele representarse en las estructuras clásicas, en este diseño se ha prescindido de un data staging área, decisión la cual se desarrolla al final de este apartado.

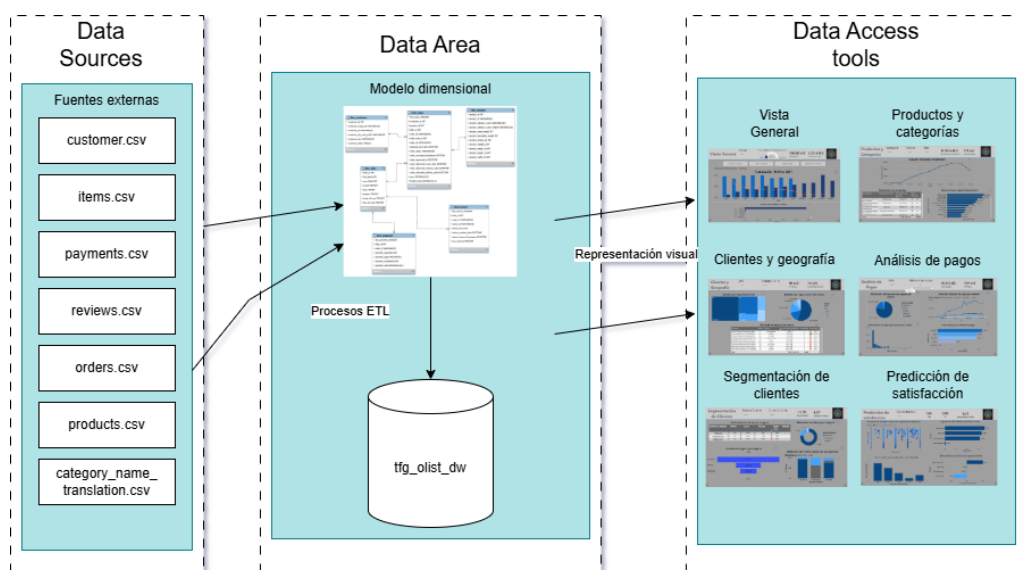


Figura 13: Arquitectura por capas del sistema desarrollado. (Elaboración propia)

4.1.1. Data Sources

Es el primer área del sistema, la base de el sistema desarrollado, que contiene toda la información del sistema origen. Está compuesto por siete archivos CSV que componen el Brazilian E-Commerce Public Dataset de Olist, obtenidos de la plataforma Kaggle. Estos archivos contienen el ciclo de vida de los pedidos realizados entre 2016 y 2018 en mercado de comercio electrónico brasileño, incluyendo todo tipo de información al respecto.

Al tratarse de un dataset de público acceso y estático, las fuentes de datos no requieren de ningún mecanismo de conexión o extracción, sino que se encuentran almacenados en el sistema de ficheros local y se accede a ellos directamente desde los scripts de los procesos ETL. A partir de estas características como la naturaleza estática del sistema y la ausencia de cargas es por lo que se decide la ausencia de implementar un área de staging, cómo se explicará más adelante.

4.1.2. Data Area

El Data Area o área de datos es el núcleo de este sistema, completamente imprescindible, y está formado por dos elementos clave con una gran relación entre ellos: el proceso ETL, implementado a través de Python y el Data Warehouse, que contiene el modelo dimensional diseñado, construido sobre MySQL. Estos elementos colaboran para transformar los datos previamente mencionados en un sistema adaptado para el análisis y la toma de decisiones.

En primer lugar, el proceso ETL o extracción, transformación y carga se ha implementado mediante 6 scripts independientes, es decir, uno por cada tabla existente en el modelo dimensional. Cada script, cada uno orientado a sus necesidades, se ha diseñado con la misma estructura dividida en tres fases. En la fase de extracción, los archivos csv almacenados localmente se leen mediante librerías pandas, que proporciona las herramientas necesarias para el tratamiento de grandes volúmenes de datos. En la fase de transformación se aplican las operaciones necesarias para el archivo, como limpieza de espacios en blanco, eliminación de duplicados o gestión de nulos entre otros, lo que será explicado con más detalle en el apartado de implementación. Finalmente, en la fase de carga, los datos transformados se insertan al Data Warehouse mediante operaciones de inserción masiva, con una previa eliminación de los registros anteriores para evitar posibles problemas.

En segundo lugar, el Data Warehouse se ha diseñado sobre MySQL bajo el nombre `tfg_olist_dw`, siguiendo un modelo dimensional de tipo esquema en galaxia que integra tres procesos de negocio diferenciados: ventas, pagos y reseñas. Este modelo está compuesto por tres tablas de hechos, que corresponden a los procesos de negocio (`fact_sales`, `fact_payment` y `fact_review`) y tres dimensiones compartidas (`dim_customer`, `dim_product` y `dim_date`). Las tablas de hechos se relacionan con las dimensiones a través de claves sustitutas, que se han decidido utilizar para desacoplar el sistema de los identificadores originales del dataset y mejorar la consistencia del modelo analítico. Asimismo, las distintas tablas de hechos se encuentran relacionadas de forma indirecta mediante el identificador de pedido (`order_id`), lo que permite conectar los distintos procesos de negocio sin necesidad de duplicar dimensiones en cada una de ellas, manteniendo la coherencia del modelo.

4.1.3. Data Access Tools

Para este proyecto se ha decidido que la capa de acceso y visualización se diseñe utilizando Microsoft Power BI Desktop, que actúa como interfaz analítica sobre el Data Warehouse. Power BI se conecta a la base de datos de MySQL a través de su conector nativo, cargando toda la información para poder realizar su análisis y representarlo visualmente. La representación visual se organiza en cinco páginas, cada una orientada a un proceso de negocio o perspectiva de análisis diferente. Estas páginas se han definido como: Visión general, Productos y categorías, Clientes y geografía, Análisis de pago y Segmentación de clientes.

Cada página contiene información a través de KPIs y gráficos de distintas características. Además, dispone de filtros interactivos que afectan a las visualizaciones, permitiendo al usuario analizar el sistema desde distintas perspectivas sin necesidad de conocimientos técnicos, al mismo tiempo que permite a los usuarios más técnicos realizar análisis con mayor profundidad. Finalmente, la navegación entre las distintas páginas se facilita mediante botones visibles en todas las vistas, y cada gráfico

contiene un botón de información para poder obtener una información mas detallada sobre lo que está siendo analizado, dotando así al informe de una experiencia de uso coherente y fluida.

4.1.4. Justificación de la ausencia de Área de Staging

En arquitecturas de Data Warehouse empresariales es muy común incorporar un área de staging intermedia entre las fuentes de datos originales y el Data Warehouse definitivo. Esta zona permite almacenar los datos brutos sin transformar antes de realizar los procesos ETL, lo que permite, entre otras cosas, la recuperación de datos en casos de fallo.

A la hora de analizar el diseño de este proyecto, se ha optado por la ausencia de esta capa por las siguientes razones:

- **Fuente de datos estática y única.** El dataset de Olist utilizado es un conjunto de archivos CSV cerrado, que no va a experimentar actualizaciones. En un entorno con cargas diarias, el staging sería necesario para aislar los nuevos datos entrantes de los que ya están procesados.
- **ETL en memoria como staging funcional.** A pesar de no tener el peso de un área de staging, los DataFrames de `pandas` actúan como una capa parecida en memoria. Almacenan temporalmente los datos crudos extraídos de los archivos antes de aplicar los procesos ETL. Por tanto, en el proyecto este staging en memoria es funcionalmente equivalente al staging persistido en base de datos.
- **Simplicidad arquitectónica justificada.** En el contexto de este proyecto con una fuente de origen única y con carga completa, añadir una zona de staging habría incrementado la complejidad del sistema, al requerir una base de datos extra y scripts de sincronización, sin aportar un valor adicional. La arquitectura resultante de tres capas es más sencilla, fácil de mantener y suficiente para el objetivo de este proyecto.

A pesar de estas razones cabe destacar que, en futuras actualizaciones, esta arquitectura podría ser adapta para permitir el análisis de nuevos datos y cargas incrementales sin necesidad de rediseñar todo el proyecto.

4.2. Diseño del modelo dimensional

En este apartado se explican las decisiones de diseño en el modelado dimensional.

4.2.1. Metodología de diseño: Enfoque Kimball

El diseño del modelo dimensional se ha realizado siguiendo la metodología propuesta por Ralph Kimball, basada en un enfoque bottom-up y orientada al análisis del negocio. Esta metodología, como se ha mencionado anteriormente, se basa en el uso de modelos dimensionales compuestos por tablas de hechos y dimensiones, con el objetivo de facilitar la consulta y explotación de los datos [4].

Entre los principios claves definidos por esta metodología, podemos destacar los siguientes que han sido utilizados en el proyecto:

- La definición explícita de la granularidad de cada tabla de hechos antes de su diseño.
- El uso del esquema en estrella como base conceptual para facilitar las consultas analíticas. Cabe destacar que el modelo en estrella es la base, mientras que la solución proporcionada esta compuesta por varios modelos en estrella al existir mas de una tabla de hechos.
- La separación existente entre hechos, eventos medibles y cuantificables, como los pagos o las ventas, y dimensiones, que aportan contexto descriptivo a estos eventos, como pueden ser las fechas o los productos.
- La utilización de claves sustitutas en todas las dimensiones para desacoplar del sistema origen, cuyo uso será argumentado en un punto posterior.

Este enfoque y estas decisiones de diseño han permitido construir un modelo flexible, fácilmente escalable y preparado para su análisis.

4.2.2. Tipo de esquema: Galaxy Schema

Aunque como ya ha sido mencionado el proyecto ha sido desarrollado tomando como base el modelo en estrella propuesto por Kimball, la existencia de tres procesos de negocio diferenciados para el análisis, que son ventas, pagos y reseñas, ha producido que se adapte a un modelo con múltiples tablas de hechos que comparten dimensiones comunes. Este tipo de esquema recibe el nombre de esquema en galaxia (Galaxy Schema) o constelación de hechos.

La utilización de este enfoque permite analizar cada proceso de negocio de forma independiente, al mismo tiempo que mantiene una visión global mediante el uso de dimensiones compartidas. De esta manera se evita la duplicidad de la información y se garantiza la coherencia en el análisis, facilitando la comparación entre indicadores procedentes de distintas tablas de hechos.

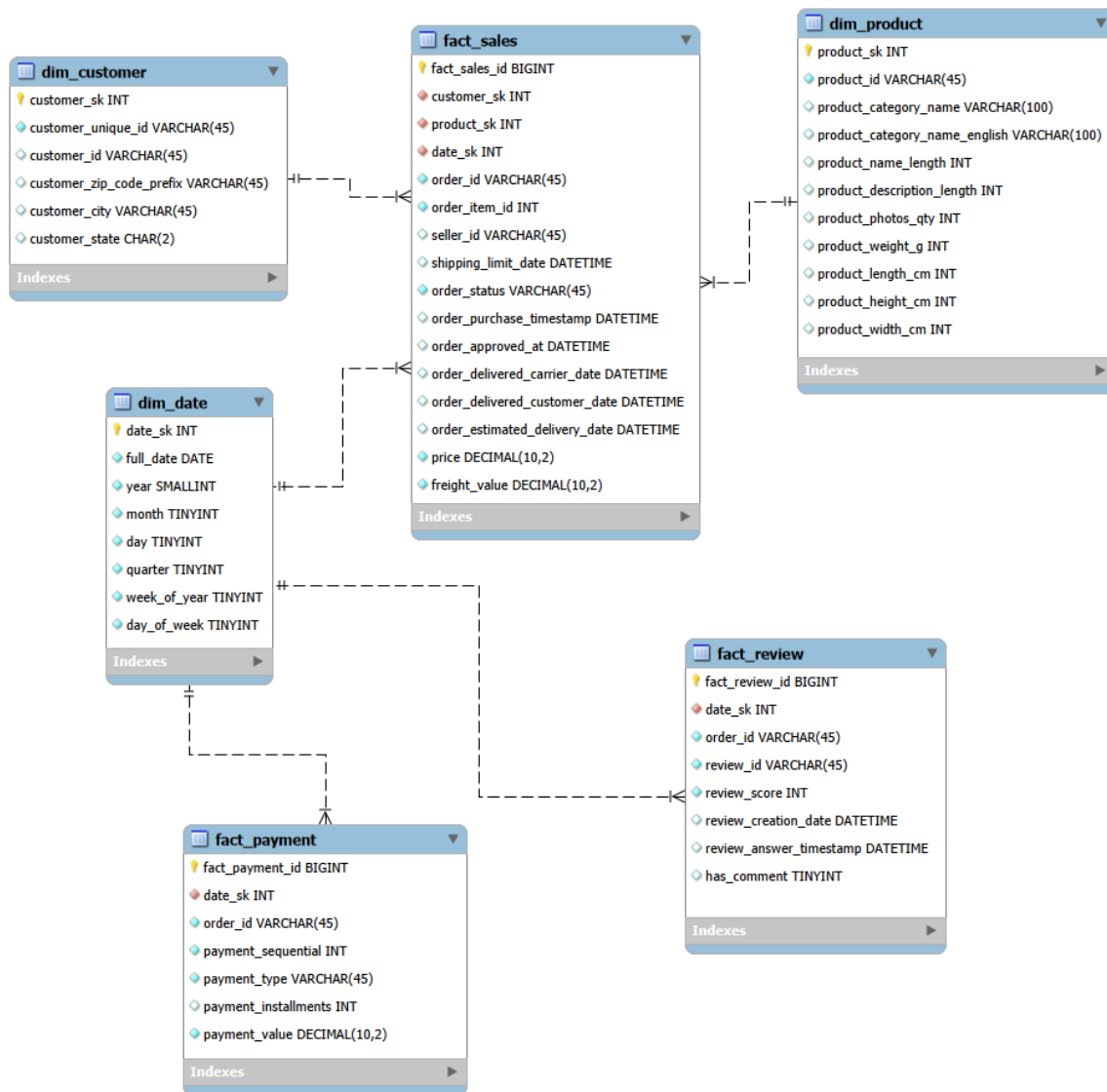


Figura 14: Modelo dimensional del sistema basado en un esquema en galaxia. (Elaboración propia)

4.2.3. Resumen de tablas del modelo

El modelo dimensional está compuesto por seis tablas: tres dimensiones y 3 hechos. En la siguiente tabla se resume la información correspondiente a cada una:

Tabla	Tipo	Granularidad	Descripción
<code>dim_customer</code>	Dimensión	-	Tabla de clientes, contiene un registro por cliente único (<code>customer_unique_id</code>).
<code>dim_product</code>	Dimensión	-	Tabla de productos, contiene un registro por producto del catálogo.
<code>dim_date</code>	Dimensión	-	Tabla de fechas, contiene un registro por día del periodo 2016–2018.
<code>fact_sales</code>	Hecho	Línea de pedido	Tabla de ventas, contiene un registro por producto dentro de un pedido.
<code>fact_payment</code>	Hecho	Pago de pedido	Tabla de pagos, contiene un registro por pago secuencial de un pedido.
<code>fact_review</code>	Hecho	Reseña	Tabla de reseñas, contiene un registro por valoración de un pedido.

Cuadro 1: Resumen de tablas del modelo dimensional

4.2.4. Decisiones generales de diseño

Durante el diseño del modelo se han adoptado las siguientes decisiones clave:

- Se consideran ventas válidas únicamente aquellos pedidos cuyo estado se encuentra en **delivered**, garantizando que los análisis de facturación se basen únicamente en pagos completados.
- La fecha principal de análisis es la fecha de compra (`order_purchase_timestamp`), a partir de la cual deriva la clave de fecha utilizada en las tablas de hechos. Esto permite que todos los análisis se realicen en torno a una misma dimensión temporal.
- La granularidad del hecho principal se ha definido como una línea de pedido, lo que significa que hay un registro por cada producto dentro de un pedido, permitiendo de esta manera calcular métricas a distintos niveles como cliente, producto o pedido.
- El modelo se desacopla de los identificadores del sistema origen mediante el uso de claves sustitutas en todas las dimensiones. A pesar de esto, las claves originales del dataset se almacenan como atributos para mantener la trazabilidad.
- En cuanto a la disposición del modelo, podemos observar que las tablas tienen una conexión indirecta a través del atributo `order_id`, permitiendo relacionar los procesos de ventas, pagos y valoraciones sin necesidad de duplicar dimensiones en cada una. Esta decisión de diseño permite

que se mantenga la independencia de cada hecho al mismo tiempo que se reduce la redundancia, garantizando la coherencia analítica. Además, cabe destacar que todas las tablas comparten la dimensión temporal, facilitando el análisis desde la perspectiva temporal.

4.2.5. Uso de claves sustitutas (Surrogate Keys)

Aunque el proyecto integra datos procedentes de una única fuente, el dataset de Olist, se ha optado por el uso de claves sustitutas (SK) como claves primarias en las dimensiones.

Esta decisión de diseño se justifica por las siguientes razones:

- Permiten desacoplar el modelo analítico del sistema origen, evitando las dependencias directas a los identificadores originales del dataset.
- Facilitan la homogeneidad del diseño, empleando el mismo patrón de clave primaria en todas las tablas de dimensiones.
- Mejoran la claridad conceptual del Data Warehouse, diferenciando explícitamente entre identificadores de negocio y claves internas del modelo.
- Permiten gestionar de forma adecuada los cambios en las dimensiones a lo largo del tiempo (Slowly Changing Dimensions), manteniendo el histórico de los datos sin afectar a la integridad del modelo.
- Preparan el modelo para una posible integración futura de nuevas fuentes de datos sin necesidad de rediseñar las relaciones existentes.

4.2.6. Diseño tablas dimensión

Atributo	Tipo	Restricciones	Descripción
customer_sk	INT	PK, NOT NULL, AI	Clave sustituta de la dimensión. Identifica de forma única cada registro del cliente dentro del Data Warehouse.
customer_unique_id	VARCHAR(45)	NOT NULL, UNIQUE	Identificador único del cliente en el sistema origen. Se conserva para trazabilidad.
customer_id	VARCHAR(45)	NOT NULL	Identificador del cliente asociado al pedido en el sistema origen.
customer_zip_code_prefix	VARCHAR(45)	NULL	Código postal del cliente.
customer_city	VARCHAR(45)	NULL	Ciudad del cliente.
customer_state	CHAR(2)	NULL	Estado del cliente.

Cuadro 2: Tabla dim_customer

Atributo	Tipo	Restricciones	Descripción
product_sk	INT	PK, NOT NULL, AI	Clave sustituta de la dimensión. Identifica de forma única cada producto en el modelo analítico.
product_id	VARCHAR(45)	NOT NULL, UNIQUE	Identificador del producto en el sistema origen. Se conserva para trazabilidad.
product_category_name	VARCHAR(100)	NULL	Categoría original del producto.
product_category_name_english	VARCHAR(100)	NULL	Categoría del producto traducida al inglés. Facilita el análisis.
product_name_length	INT	NULL	Longitud del nombre del producto.
product_description_length	INT	NULL	Longitud de la descripción del producto.
product_photos_qty	INT	NULL	Número de fotografías asociadas al producto.
product_weight_g	INT	NULL	Peso del producto en gramos.
product_length_cm	INT	NULL	Longitud del producto en centímetros.
product_height_cm	INT	NULL	Altura del producto en centímetros.
product_width_cm	INT	NULL	Anchura del producto en centímetros.

Cuadro 3: Tabla dim_product

Atributo	Tipo	Restricciones	Descripción
date_sk	INT	PK, NOT NULL	Clave sustituta de la dimensión temporal. Se genera en formato numérico para facilitar uniones y agregaciones.
full_date	DATE	UNIQUE, NOT NULL	Fecha completa asociada a la clave temporal.
year	SMALLINT	NOT NULL	Año de la fecha.
month	TINYINT	NOT NULL	Mes de la fecha.
day	TINYINT	NOT NULL	Día del mes.
quarter	TINYINT	NOT NULL	Trimestre del año.
week_of_year	TINYINT	NOT NULL	Semana del año.
day_of_week	TINYINT	NOT NULL	Día de la semana.

Cuadro 4: Tabla dim_date

4.2.7. Diseño tablas hecho

Atributo	Tipo	Restricciones	Descripción
fact_sales_id	BIGINT	PK, NOT NULL, UNIQUE, AI	Clave sustituta de la tabla de ventas. Identifica de forma única cada línea de venta.
customer_sk	INT	FK, NOT NULL	Clave foránea hacia dim_customer . Permite analizar ventas por cliente y localización.
product_sk	INT	FK, NOT NULL	Clave foránea hacia dim_product . Permite analizar ventas por producto y categoría.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite un análisis temporal de las ventas.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido.
order_item_id	INT	NOT NULL	Identificador del artículo dentro del pedido.
seller_id	VARCHAR(45)	NULL	Identificador del vendedor.
shipping_limit_date	DATETIME	NULL	Fecha límite del envío del producto.
order_status	VARCHAR(45)	NOT NULL	Estado del pedido.
order_purchase_timestamp	DATETIME	NOT NULL	Fecha y hora de compra del pedido.
order_approved_at	DATETIME	NULL	Fecha de aprobación del pedido. Puede ser nula si no consta en origen.
order_delivered_carrier_date	DATETIME	NULL	Fecha de entrega del pedido al transportista.
order_delivered_customer_date	DATETIME	NULL	Fecha de entrega al cliente.
order_estimated_delivery_date	DATETIME	NULL	Fecha estimada de entrega al cliente.
price	DECIMAL(10,2)	NOT NULL	Importe del producto vendido.
freight_value	DECIMAL(10,2)	NOT NULL	Coste de envío del producto.

Cuadro 5: Tabla **fact_sales**

Atributo	Tipo	Restricciones	Descripción
fact_payment_id	BIGINT	PK, NOT NULL, AI	Clave sustituta de la tabla de hechos de pagos. Identifica de forma única cada registro de pago.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite analizar los pagos desde un punto de vista temporal.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido asociado al pago.
payment_sequential	INT	NOT NULL	Secuencia del pago dentro de un mismo pedido.
payment_type	VARCHAR(45)	NOT NULL	Método de pago utilizado por el cliente.
payment_installments	INT	NOT NULL	Número de cuotas del pago.
payment_value	DECIMAL(10,2)	NOT NULL	Importe del pago.

Cuadro 6: Tabla **fact_payment**

Atributo	Tipo	Restricciones	Descripción
fact_review_id	BIGINT	PK, NOT NULL, AI	Clave sustituta de la tabla de hechos de valoraciones. Identifica de forma única cada valoración.
date_sk	INT	FK, NOT NULL	Clave foránea hacia dim_date . Permite analizar las valoraciones desde un punto de vista temporal.
order_id	VARCHAR(45)	NOT NULL	Identificador del pedido valorado.
review_id	VARCHAR(45)	NOT NULL	Identificador de la valoración en el sistema origen.
review_score	INT	NOT NULL	Puntuación otorgada por el cliente del 1 al 5.
review_creation_date	DATETIME	NOT NULL	Fecha de creación de la valoración.
review_answer_timestamp	DATETIME	NULL	Fecha de respuesta de la valoración.
has_comment	TINYINT	NOT NULL	Indicador 0/1 que refleja si la valoración incluye comentario.

Cuadro 7: Tabla **fact_review**

4.3. Diseño de los dashboards

En este apartado se describe el diseño previo a la implementación de los dashboards en Power BI. Se recogen las decisiones que se han tomado respecto a la estructura general del informe, relativas a factores como la disposición de los elementos en cada página, los colores utilizados y el sistema de navegación de páginas, entre otros.

4.3.1. Boceto previo

Con carácter previo a la implementación de las páginas, se elaboró un boceto general que recoge la disposición estructural de los elementos comunes a todas las páginas. Cabe destacar que es un

boceto previo por lo que cada página posteriormente se adapta a sus necesidades. Este boceto sirvió como referencia durante la fase de implementación, permitiendo tomar decisiones de diseño de forma anticipada y garantizando la coherencia en el resultado final.

El boceto define una plantilla común aplicable a las seis páginas del informe, estableciendo las siguientes zonas funcionales: una franja superior destinada a los indicadores claves de negocio (KPIs) y los filtros interactivos, una zona central para las representaciones visuales de los análisis, en la que cada página se adapta con el número de gráficos o representaciones que se considera necesario, y una serie de elementos interactivos para facilitar la navegación y comprensión del sistema. Esta estructura permite al usuario que pueda orientarse con facilidad, al encontrar siempre los elementos en la misma posición.



Figura 15: Boceto inicial de la estructura de los cuadros de mando. (Elaboración propia)

4.3.2. Principios de diseño aplicados

El diseño de los distintos dashboards desarrollados se ha guiado por los siguientes principios, aplicados en todas las páginas:

- **Claridad y legibilidad.** En primer lugar, los dashboards se separan por categorías, de manera que los usuarios pueden guiarse con claridad sobre las distintas pantallas. Además, cada visualización en los informes transmite un único mensaje claro, prefiriendo múltiples visualizaciones simples a una sola visualización más compleja de interpretar.
- **Jerarquía visual.** Se aplican los principios de jerarquía visual en los que la parte superior de un dashboard capta la atención principal de los usuarios. Por tanto, los indicadores principales (KPIs de negocio) se sitúan en la parte superior, con tipografía de mayor tamaño y peso visual, al igual que el nombre de la página y los filtros disponibles, permitiendo que el usuario identifique de inmediato los valores clave antes de analizar en detalle las visualizaciones inferiores.

- **Consistencia.** Se aplica el mismo diseño en todas las páginas, teniendo en cuenta factores como la paleta de colores, la tipografía o la disposición de los elementos. Esto reduce la carga para el usuario, que no necesita reaprender la interfaz cada vez que cambia la pantalla.
- **Interactividad controlada.** En cada página se incorporan únicamente filtros relevantes para el contenido, no exponiendo a los usuarios a opciones que no modifiquen las visualizaciones existentes. Cabe destacar que, principalmente en el caso de la representación de las técnicas de análisis avanzado, se presentan algunos gráficos fijos que representan datos no variables. Los filtros, como se ha indicado, se encuentran en la parte superior para que su peso visual se vea potenciado.
- **Orientación al usuario de negocio.** El informe está diseñado para ser entendido por todo tipo de usuario, tanto técnico como no técnico. Los títulos son descriptivos y en lenguaje natural, los valores vienen detallados, entre otras cosas. Además de esto, se incorporan iconos de información en cada gráfico para permitir acceder a información más detallada.

4.3.3. Identidad visual

La identidad visual del informe se definió en la fase de diseño previo a la implementación, y se mantiene en todo el informe.

En primer lugar, respecto a la paleta de colores, se han utilizado diferentes tipos dependiendo de los objetivos. En cuanto a los colores de los gráficos, se basa en una escala de diferentes tonos de azules, ya que es el color asociado convencionalmente a los entornos de negocio y análisis de datos, ofreciendo buen contraste sobre el fondo de la imagen y permitiendo distinguir representaciones de datos mediante variaciones de saturación e intensidad.

Por otro parte, para el fondo se designa una variación del color gris, la parte superior una parte mas clara y, la inferior, un poco mas oscura, lo que potencia el contraste frente a los gráficos de color azul. Además, estos colores reducen la fatiga visual en sesiones de análisis prolongadas y aportan un aspecto profesional.

Por último, se reservan los colores rojo, amarillo y verde como indicadores, ya que son unos valores visuales comúnmente utilizados y que el usuario asocia fácilmente con su significado. También se utiliza el naranja, como caso puntual, para representar la diferencia de facturación entre dos gráficos.

En cuanto a la tipografía, se utiliza una misma familia Segoe UI, variando este tipo junto Segoe UI Semibold, utilizando variaciones de tamaño y peso para establecer la jerarquía visual. De esta manera, se designa la tipografía semibold para títulos de gráficos, filtros y KPIs, mientras que el uso normal se utiliza para valores con menor peso visual como valores o leyendas.

4.3.4. Selección de visualizaciones

Una de las decisiones de diseño más relevantes en este ámbito es la elección del tipo de visualización adecuado para cada dato y objetivo. Una elección inadecuada puede dificultar la interpretación por parte del usuario o incluso llevarle a conclusiones erróneas. La siguiente tabla recoge los tipos de visualización utilizados para cada informe y su correspondiente justificación:

Visión General		
Información elegida	Visualización elegida	Justificación
Comparativo temporal entre dos años	Gráfico de columnas agrupadas y de líneas	Permite comparar volúmenes absolutos y la tendencia relativa en un mismo gráfico.
Estado del pedido	Gráfico de barras agrupadas	Permite comparar de forma clara la distribución de pedidos según su estado, identificando rápidamente las categorías predominantes.

Cuadro 8: Justificación de las visualizaciones del dashboard de Visión General

Productos y Categorías		
Información elegida	Visualización elegida	Justificación
Evolución temporal de la facturación	Gráfico de líneas	Se selecciona un gráfico de líneas ya que permite representar de manera clave la evolución de una variable a lo largo del tiempo, facilitando la detección de tendencias y posibles alteraciones en el comportamiento.
Rendimiento de productos	Tabla con indicadores visuales	Se emplea una tabla debido a que permite incluir múltiples atributos asociados a un producto simultáneamente. Los indicadores visuales permiten identificar rápidamente la tendencia de los productos.
Facturación por categoría de producto	Gráfico de barras agrupadas	Se selecciona un gráfico de barras agrupadas porque permite calcular de forma clara la facturación entre categorías, facilitando la identificación de las categorías con mayor facturación. Además mejora la legibilidad de etiquetas contextuales largas.

Cuadro 9: Justificación de las visualizaciones del dashboard de Productos y Categorías

Clientes y Geografía		
Información elegida	Visualización elegida	Justificación
Facturación por estado	Treemap	Se selecciona un mapa de árbol ya que resulta innovador, alejándose de los típicos gráficos. También aporta una visión rápida de las regiones con mayor peso económico.
Distribución de clientes por estado	Gráfico circular	Se utiliza un gráfico circular para representar la proporción de clientes por estado, permitiendo comparar el peso relativo de cada categoría sobre el total.
Actividad de compra por cliente	Tabla con indicadores visuales	Se emplea una tabla con indicadores visuales para mostrar simultáneamente diferentes atributos asociados a cada cliente, facilitando la identificación de patrones. Los patrones visuales permiten detectar si un cliente es recurrente o no, factor importante para decisiones de negocio.

Cuadro 10: Justificación de las visualizaciones del dashboard de Clientes y Geografía

Análisis de pagos		
Información elegida	Visualización elegida	Justificación
Distribución del importe total pagado por método	Gráfico circular	Se selecciona un gráfico circular debido a que permite visualizar la proporción que representa cada método de pago sobre el importe total procesado.
Evolución temporal de pagos por método	Gráfico de líneas	Se emplea un gráfico de líneas múltiples para representar la evolución temporal de distintos métodos y comparar su comportamiento.
Distribución de pagos por número de cuotas	Gráfico de columnas apiladas	Se utiliza un gráfico de columnas apiladas que permite ver la frecuencia de aparición de los diferentes números de cuotas, identificando concentraciones y patrones de comportamiento.
Ticket medio por método de pago	Gráfico de barras agrupadas	Se utiliza un gráfico de barras agrupadas para comparar el valor medio del ticket de los diferentes métodos de pago e identificar diferencias significativas.

Cuadro 11: Justificación de las visualizaciones del dashboard de Análisis de pagos

Segmentación de clientes		
Información elegida	Visualización elegida	Justificación
Perfil medio de clientes por categoría	Tabla con indicadores visuales	Se emplea una tabla con indicadores visuales para mostrar visualmente distintas métricas asociadas a cada segmento en el proceso de clustering, permitiendo una comparación entre sectores.
Distribución de clientes por categoría	Gráfico de anillos	Se selecciona un gráfico de anillos ya que aporta una alternativa al gráfico circular y representa la proporción relativa de clientes pertenecientes a cada segmento de forma clara y compacta.
Contribución al gasto por categoría	Embudo	En este caso se utiliza un gráfico de embudo para representar la contribución relativa de cada segmento al gasto total, permitiendo visualizar de forma intuitiva la reducción progresiva entre categorías.
Distribución del nivel de satisfacción por segmento	Gráfico de columnas 100 % apiladas	Se utiliza un gráfico de columnas 100 % apiladas para comparar la composición porcentual de los niveles de satisfacción dentro de cada segmento.

Cuadro 12: Justificación de las visualizaciones del dashboard de Segmentación de clientes

Predicción de satisfacción		
Información elegida	Visualización elegida	Justificación
Precisión del modelo: valoración real frente a estimada	Gráfico de dispersión	Se utiliza un gráfico de dispersión para analizar la relación entre los valores reales y las predicciones del modelo, lo que permite evaluar visualmente su precisión.
Influencia de variables en la satisfacción (Random Forest)	Gráfico de barras apiladas	Se selecciona un gráfico de barras apiladas para representar la importancia relativa de cada variable en el modelo Random Forest, facilitando la identificación de los factores con mayor influencia.
Efecto de las variables en la regresión lineal	Gráfica de barras agrupadas	Se utiliza un gráfico de barras agrupadas para representar el sentido y magnitud de los coeficientes del modelo, diferenciando claramente impactos positivos y negativos.
Error medio según valoración real del cliente	Gráfico de columnas agrupadas	Se selecciona un gráfico de columnas agrupadas para comparar el error medio del modelo según el nivel de valoración real, facilitando la identificación de rangos donde la predicción presenta mayor desviación.

Cuadro 13: Justificación de las visualizaciones del dashboard de Predicción de satisfacción

Además de todas estas visualizaciones implementadas en los dashboards, también se incorporan otros elementos complementarios ya mencionados para facilitar la interpretación y mejorar la interacción.

En cuanto a los KPIs, se representan principalmente mediante tarjetas simples, las cuales muestran de forma directa el valor asociado a una métrica relevante, como puede ser la facturación total. Asimismo, en alguna ocasión también se ha optado por la idea de utilizar un medidor, que permite representar visualmente el valor alcanzado respecto a un rango objetivo o valor de referencia.

Por otro lado, para proporcionar capacidad de exploración y análisis dinámico se utilizan los filtros, representado con el nombre segmentación de datos. Su implementación puede adoptar distintos formatos, entre los que se han elegido los menús desplegables, utilizados para seleccionar valores concretos de una categoría, y los filtros de tipo entre, empleados para valores numéricos, como el número de cuotas. Estos últimos funcionan mediante un marcador deslizante que se mueve entre dos valores límite, permitiendo definir de forma visual el intervalo de valores sobre el que se realiza el análisis.

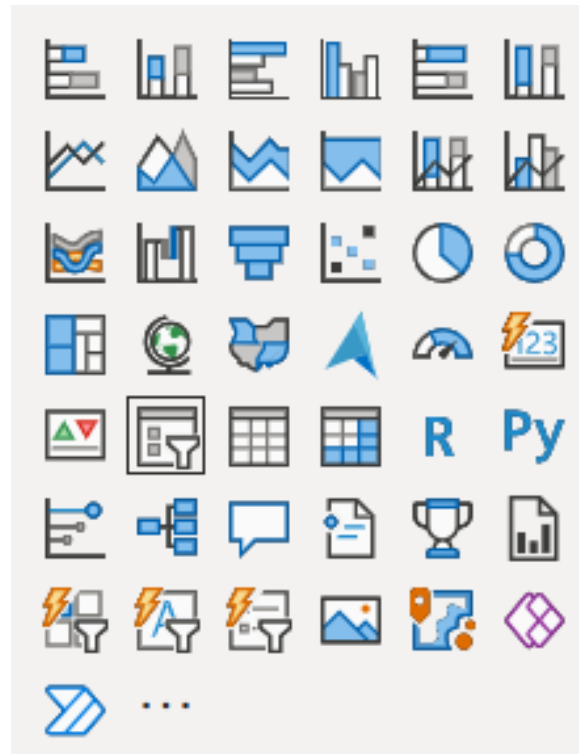


Figura 16: Visualizaciones disponibles en Power BI para la construcción de cuadros de mando.[15]

4.3.5. Sistema de navegación

El diseño de la navegación entre páginas se diseñó en la fase previa a la implementación, con el objetivo de que el usuario pueda moverse entre las distintas páginas del informe sin complejidad. Para ello se diseñó en primer lugar, en la página inicial, una barra de navegación superior con botones con el nombre de cada página que actúan como accesos directos. Además de este sistema, las posteriores páginas contienen un sistema basado en flechas, una situada en la esquina inferior izquierda y otra en la esquina inferior derecha, lo que permite al usuario navegar por el sistema con facilidad.

Esta decisión responde en primer lugar al principio de navegación no lineal, permitiendo acceder desde la primera página a todas sin necesidad de seguir un orden predeterminado. Por otra parte, también permite a los usuarios una navegación lineal adoptando el orden que se ha considerado adecuado en el diseño del informe. De esta manera, el usuario puede adoptar la técnica que mas le convenga sin necesidad de seguir unas pautas fijas.

5. Implementación de la solución desarrollada

En este punto, tras presentar la fase de diseño, se describe la implementación técnica de las distintas partes del sistema de Business Intelligence. Se aborda todo el proceso de implementación, desde el desarrollo de los scripts ETL sobre los datos del sistema origen hasta la representación visual de los datos.

5.1. Integración y preparación de datos (ETL)

Este apartado describe la implementación del proceso ETL que alimenta el Data Warehouse con los datos del dataset de Olist. El proceso se implementa en Python mediante scripts independientes, uno por cada tabla del modelo. En este apartado se va a explicar el desarrollo de cada script, pero cabe destacar un patrón común entre todos, la configuración. La contraseña de acceso a MySQL no se almacena en el código fuente sino que se lee en tiempo de ejecución desde la variable de entorno del sistema operativo `MYSQL_PASSWORD`, garantizando que las credenciales no queden expuestas en el repositorio del proyecto.

```
DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": os.getenv("MYSQL_PASSWORD"),
    "database": "tfg_olist_dw",
    "port": 3306
}
```

Figura 17: Configuración de conexión entre el proceso ETL y el Data Warehouse. (Elaboración propia)

5.1.1. `load_dim_customer.py`

Este script carga la dimensión de clientes. El principal reto es que el dataset original contiene múltiples filas para un mismo cliente, ya que cada pedido genera un registro independiente con el mismo `customer_unique_id` pero distinto `customer_id`, que refleja el indicador de sesión de compra.

Tabla <code>load_dim_customer.py</code>	
Tabla destino	<code>dim_customer</code>
CSV de entrada	<code>olist_customers_dataset.csv</code>
Dependencias	Ninguna
Registros aprox.	96096 clientes únicos (99441 originales)

Cuadro 14: Resumen del script `load_dim_customer.py`

En primer lugar, en la fase de extracción se carga el archivo `olist_customers_dataset.csv` en un único DataFrame. Como se ha mencionado anteriormente, este archivo contiene una fila por cada pedido realizado y no por cada cliente único, por lo que un mismo cliente puede aparecer múltiples veces dentro del conjunto de datos.

```
CUSTOMERS_CSV = "olist_customers_dataset.csv"
df = pd.read_csv(CUSTOMERS_CSV)
```

Figura 18: Fragmento de código correspondiente a la fase de extracción de `dim_customer`. (Elaboración propia)

En cuanto a la transformación, el script aplica tres operaciones:

- **Limpieza de strings.** Los campos `customer_id`, `customer_unique_id`, `customer_city` y `customer_state` se convierten a tipo string y se eliminan los espacios en blanco extremos mediante `.astype(str).str.strip()` evitando inconsistencias y posibles duplicados invisibles durante el proceso posterior de deduplicación.
- **Deduplicación.** Se aplica `drop_duplicates()` sobre el atributo `customer_unique_id`, conservando únicamente la primera aparición de cada cliente real. Esta operación reduce el DataFrame original manteniendo un único registro por cliente y eliminando filas adicionales asociadas a pedidos posteriores realizados por el mismo usuario.
- **Selección de columnas.** Se conservan únicamente los atributos relevantes que formarán parte de la dimensión, seleccionando los campos que posteriormente serán cargados en la tabla `dim_customer`.

```
# =====
# LIMPIEZA BÁSICA
# =====

for col in ["customer_id", "customer_unique_id", "customer_city", "customer_state"]:
    df[col] = df[col].astype(str).str.strip()

# =====
# DEDUPLICAR PARA DIMENSIÓN
# =====

df_dim = df.drop_duplicates(subset=["customer_unique_id"], keep="first").copy()

# =====
# SELECCIONAR COLUMNAS QUE CARGAMOS EN dim_customer
# =====
df_dim = df_dim[[
    "customer_unique_id",
    "customer_id",
    "customer_zip_code_prefix",
    "customer_city",
    "customer_state"
]]
```

Figura 19: Fragmento de código correspondiente a la fase de transformación de `dim_customer`. (Elaboración propia)

Finalmente, durante el proceso de carga se establece conexión con la base de datos del Data Warehouse y se ejecuta una operación `DELETE` sobre la tabla destino para evitar inconsistencias derivadas de cargas anteriores. Tras ello, los registros se insertan mediante `executemany()`, optimizando la carga masiva de datos, y finalmente se cierran las conexiones utilizadas.

```

# =====
# CONECTAR A MYSQL
# =====
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()

# =====
# VACIAR TABLA (RECARGA COMPLETA)
# =====
cur.execute("DELETE FROM dim_customer;")
conn.commit()

# =====
# 8) INSERT MASIVO
# =====
insert_sql = """
    INSERT INTO dim_customer
    (customer_unique_id, customer_id, customer_zip_code_prefix, customer_city, customer_state)
    VALUES (%s, %s, %s, %s, %s)
"""

data = list(df_dim.itertuples(index=False, name=None))

cur.executemany(insert_sql, data)
conn.commit()

# =====
# 9) CERRAR CONEXIONES
# =====
cur.close()
conn.close()

```

Figura 20: Fragmento de código correspondiente a la fase de carga de `dim_customer`. (Elaboración propia)

5.1.2. `load_dim_product.py`

Este script es el más complejo de los tres de dimensiones. Durante este proceso lee dos fuentes independientes, corrige errores tipográficos del dataset original y carga todos los datos de forma adecuada para su futuro tratamiento.

Tabla <code>load_dim_product.py</code>	
Tabla destino	<code>dim_product</code>
CSV de entrada	<code>olist_products_dataset.csv</code> , <code>product_category_name_translation.csv</code>
Dependencias	Ninguna
Registros aprox.	32951 productos

Cuadro 15: Resumen del script `load_dim_product.py`

En la fase de extracción, como se ha mencionado anteriormente, se leen en paralelo los archivos `olist_products_dataset.csv` y `product_category_name_translation.csv`. El primer archivo contiene la información asociada a los productos, incluyendo la categoría definida en su idioma original, portugués. Por otro lado, el segundo proporciona la traducción de dichas categorías al inglés.

```
PRODUCTS_CSV = "olist_products_dataset.csv"
CATEGORIES_CSV = "product_category_name_translation.csv"
products = pd.read_csv(PRODUCTS_CSV)
categories = pd.read_csv(CATEGORIES_CSV)
```

Figura 21: Fragmento de código correspondiente a la fase de extracción de `dim_product`. (Elaboración propia)

En cuanto a la transformación, se realizan las siguientes operaciones de manera secuencial:

- **Corrección de errores tipográficos.** El dataset original contiene dos columnas con la palabra *length* incorrectamente escrita como *lenght*. Este problema se corrige mediante `rename()` antes de cualquier otra operación, garantizando una nomenclatura consistente y correcta en el Data Warehouse.
- **Limpieza de campos clave.** Se aplica sobre los atributos `product_id` y `product_category_name` del DataFrame de productos, así como sobre las columnas del DataFrame de traducciones. Estos atributos se convierten a tipo string y se eliminan espacios en blanco extremos para evitar inconsistencias durante las operaciones posteriores.
- **Join con el archivo de traducción.** Se realiza un `LEFT JOIN` entre los productos y las categorías traducidas utilizando el atributo `product_category_name`. El uso de esta operación garantiza que todos los productos permanezcan en la dimensión aunque no exista una traducción asociada, en cuyo caso el atributo correspondiente quedará con valor `NULL`.
- **Deduplicación.** Se garantiza la existencia de un único registro por cada `product_id`.
- **Selección de columnas.** Se conservan únicamente los atributos necesarios para el análisis posterior: `product_id`, `product_category_name`, `product_category_name_english`, `product_name_length`, `product_description_length`, `product_photos_qty`, `product_weight_g`, `product_length_cm`, `product_height_cm` y `product_width_cm`.
- **Gestión de valores nulos.** Los valores `NaN` se transforman a `None` para permitir su correcta interpretación como `NULL` durante la inserción en MySQL. Esta conversión permite mantener registros incompletos sin necesidad de eliminarlos del proceso de carga.

```

# =====
# 2) CORREGIR COLUMNAS MAL ESCRITAS
# =====
products = products.rename(columns={
    "product_name_lenght": "product_name_length",
    "product_description_lenght": "product_description_length"
})

# =====
# 3) LIMPIEZA DE CAMPOS CLAVE
# =====
products["product_id"] = products["product_id"].astype("string").str.strip()
products["product_category_name"] = products["product_category_name"].astype("string").str.strip()

categories["product_category_name"] = categories["product_category_name"].astype("string").str.strip()
categories["product_category_name_english"] = categories["product_category_name_english"].astype("string").str.strip()
# =====
# 4) TRANSFORM → JOIN PARA AÑADIR TRADUCCIÓN
# =====
df = products.merge(
    categories,
    on="product_category_name",
    how="left"
)
# =====
# 5) ELIMINAR DUPLICADOS
# =====
df_dim = df.drop_duplicates(subset=["product_id"], keep="first").copy()

```

Figura 22: Fragmento de código correspondiente a la primera fase de transformación de `dim_product`. (Elaboración propia)

```

# =====
# 6) SELECCIONAR COLUMNAS DEL DATA WAREHOUSE
# =====
df_dim = df_dim[[
    "product_id",
    "product_category_name",
    "product_category_name_english",
    "product_name_length",
    "product_description_length",
    "product_photos_qty",
    "product_weight_g",
    "product_length_cm",
    "product_height_cm",
    "product_width_cm"
]]
# =====
# 7) CONVERTIR NaN A None
# =====
df_dim = df_dim.astype(object).where(df_dim.notna(), None)

```

Figura 23: Fragmento de código correspondiente a la segunda fase de transformación de `dim_product`. (Elaboración propia)

Finalmente, durante el proceso de carga se establece conexión con la base de datos del Data Warehouse y se ejecuta una operación **DELETE** sobre la tabla destino para evitar inconsistencias derivadas de cargas anteriores. Tras ello, los registros se insertan mediante `executemany()`, optimizando la carga masiva de datos, y finalmente se cierran las conexiones utilizadas.

```

# =====
# 9) CONEXIÓN A MYSQL
# =====
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()
# =====
# 10) RECARGA COMPLETA DE LA DIMENSIÓN
# =====
# Se eliminan los datos anteriores antes de insertar los nuevos
cur.execute("DELETE FROM dim_product;")
conn.commit()
# =====
# 11) SENTENCIA SQL DE INSERCIÓN
# =====
insert_sql = """
    INSERT INTO dim_product
    (
        product_id,
        product_category_name,
        product_category_name_english,
        product_name_length,
        product_description_length,
        product_photos_qty,
        product_weight_g,
        product_length_cm,
        product_height_cm,
        product_width_cm
    )
    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
"""
# =====
# 12) CONVERTIR DATAFRAME EN TUPLAS PARA INSERT MASIVO
# =====
data = list(df_dim.itertuples(index=False, name=None))
cur.executemany(insert_sql, data)
conn.commit()
# =====
# 13) CERRAR CONEXIÓN
# =====
cur.close()
conn.close()

```

Figura 24: Fragmento de código correspondiente a la fase de carga de `dim_product`. (Elaboración propia)

5.1.3. `load_dim_date.py`

Este script es el único que no carga datos directamente desde un archivo CSV hacia una dimensión del Data Warehouse, sino que genera la dimensión temporal de forma sintética. Para ello, utiliza el archivo `olist_orders_dataset.csv` únicamente con el objetivo de identificar el rango temporal mínimo y máximo existente en los pedidos, permitiendo construir automáticamente la secuencia completa de fechas comprendidas dentro de dicho intervalo.

Tabla load_dim_date.py	
Tabla destino	dim_date
CSV de entrada	olist_orders_dataset.csv (solo utilizado para obtener el rango temporal)
Dependencias	Ninguna
Registros aprox.	735 días

Cuadro 16: Resumen del script load_dim_date.py

En este caso, la extracción se realiza a partir del CSV de los pedidos, cargando el dataset para extraer el rango de fechas.

```
ORDERS_CSV = "olist_orders_dataset.csv"
orders = pd.read_csv(ORDERS_CSV)
```

Figura 25: Fragmento de código correspondiente a la fase de extracción de `dim_date`. (Elaboración propia)

Por otro lado, la transformación se compone de distintas fases. En primer lugar, el atributo `order_purchase_timestamp` se convierte a tipo `datetime`, obteniendo posteriormente las fechas mínima y máxima presentes en el conjunto de datos. Una vez determinado el intervalo temporal, se genera un `DataFrame` con una fila por cada día comprendido dentro del rango calculado. A partir de cada fecha se derivan los distintos atributos temporales que forman la dimensión. El campo `date_sk` adopta un formato numérico `YYYYMMDD` (por ejemplo, `20210204` para representar el 4 de febrero de 2021), actuando como clave primaria dentro de la dimensión temporal y utilizándose posteriormente como clave foránea en las tablas de hechos. Además, el atributo `day_of_week` se calcula sumando una unidad al valor devuelto por `pandas`, el cual numera los días desde cero. De esta forma se obtiene una escala comprendida entre 1 y 7, donde el valor 1 corresponde al lunes y el valor 7 al domingo.

Finalmente, el atributo `full_date` se convierte desde `datetime` a tipo `date`, garantizando un formato adecuado antes de realizar la carga en el Data Warehouse.


```
# =====
# 2) CONVERTIR A DATETIME
# =====
orders["order_purchase_timestamp"] = pd.to_datetime(
    orders["order_purchase_timestamp"], errors="coerce"
)
# =====
# 3) RANGO DE FECHAS
# =====
min_date = orders["order_purchase_timestamp"].min().date()
max_date = orders["order_purchase_timestamp"].max().date()
# =====
# 4) GENERAR CALENDARIO
# =====

dim_date = pd.DataFrame({
    "full_date": pd.date_range(start=min_date, end=max_date, freq="D")
})
dim_date["date_sk"] = dim_date["full_date"].dt.strftime("%Y%m%d").astype(int)
dim_date["year"] = dim_date["full_date"].dt.year
dim_date["month"] = dim_date["full_date"].dt.month
dim_date["day"] = dim_date["full_date"].dt.day
dim_date["quarter"] = dim_date["full_date"].dt.quarter
dim_date["week_of_year"] = dim_date["full_date"].dt.isocalendar().week.astype(int)
dim_date["day_of_week"] = dim_date["full_date"].dt.dayofweek + 1
dim_date["full_date"] = dim_date["full_date"].dt.date
```

Figura 26: Fragmento de código correspondiente a la fase de transformación de `dim_date`. (Elaboración propia)

Para concluir el script, como en los casos anteriores, se establece la conexión, se vacía la tabla y se cargan los atributos seleccionados.

```
# =====
# 5) CONEXIÓN MYSQL
# =====
# Se establece conexión con la base de datos del Data Warehouse
conn = mysql.connector.connect(**DB_CONFIG)
cur = conn.cursor()
# =====
# 6) RECARGA COMPLETA
# =====
cur.execute("DELETE FROM dim_date;")
conn.commit()
# =====
# 7) INSERT MASIVO
# =====
insert_sql = """
INSERT INTO dim_date
(date_sk, full_date, year, month, day, quarter, week_of_year, day_of_week)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
"""

data = list(dim_date[
    ["date_sk", "full_date", "year", "month", "day", "quarter", "week_of_year", "day_of_week"]
].itertuples(index=False, name=None))

cur.executemany(insert_sql, data)
conn.commit()
# =====
# 9) CERRAR CONEXIÓN
# =====
cur.close()
conn.close()
```

Figura 27: Fragmento de código correspondiente a la fase de carga de `dim_date`. (Elaboración propia)

Referencias

- [1] Dataprix. «BI Usability: evolución y tendencia. »dirección: <https://www.dataprix.com/articulo/business-intelligence/bi-usability-evolucion-y-tendencia>
- [2] M. Golfarelli y S. Rizzi, *Data Warehouse Design: Modern Principles and Methodologies*. McGraw-Hill, 2009.
- [3] TAKTIC. «Cómo se aplica el Business Intelligence. »dirección: <https://taktic.es/blog/como-se-aplica-el-business-intelligence/>
- [4] R. Kimball y M. Ross, *The Data Warehouse Toolkit*, 3.^a ed. Wiley, 2013.
- [5] IEBS Business School. «Microsoft Power BI: analítica y usabilidad. »dirección: <https://www.iebschool.com/hub/microsoft-power-bi-analitica-usabilidad/>
- [6] Wikimedia Commons. «Tableau Logo. »dirección: https://commons.wikimedia.org/wiki/File:Tableau_Logo.png
- [7] Wikimedia Commons. «Qlik Logo. »dirección: https://commons.wikimedia.org/wiki/File:Qlik_Logo.svg
- [8] Tekne DataLabs. «Herramienta Looker. »dirección: <https://teknedatalabs.com/es/herramienta-looker/>
- [9] Logo.wine. «Python Programming Language Logo. »dirección: [https://download.logo.wine/logo/Python_\(programming_language\)/Python_\(programming_language\)-Logo.wine.png](https://download.logo.wine/logo/Python_(programming_language)/Python_(programming_language)-Logo.wine.png)
- [10] Logo.wine. «R Programming Language Logo. »dirección: [https://download.logo.wine/logo/R_\(programming_language\)/R_\(programming_language\)-Logo.wine.png](https://download.logo.wine/logo/R_(programming_language)/R_(programming_language)-Logo.wine.png)
- [11] Dribbble. «D3.js Logo. »dirección: <https://dribbble.com/shots/2456036-D3-js-Logo>
- [12] Intuji. «What is MySQL? Definition, Comparison with SQL, Benefits and Features Explained. »dirección: <https://intuji.com/what-is-mysql-definition-comparison-with-sql-benefits-features-explained/>
- [13] Santander Open Academy. «Fundamentos de Power BI,» visitado 5 de mayo de 2026. dirección: <https://lms.santanderopenacademy.com/courses/408>
- [14] Olist. «Brazilian E-Commerce Public Dataset by Olist,» visitado 5 de mayo de 2026. dirección: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
- [15] Microsoft. «Power BI Desktop. »dirección: <https://powerbi.microsoft.com/es-es/desktop/>